"Develop a passion for learning."

MODULE 3

OBJECT-ORIENTED PROGRAMMING IN JAVA

From core OOP principles to SOLID, design patterns, and real domain modeling.

JAVA SOFTWARE ENGINEER BOOTCAMP

TEKsystems Global Services

WORKFORCE DEVELOPMENT

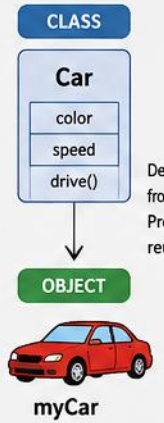
OBJECT ORIENTED PROGRAMMING (OOP) IN JAVA

OOP is a programming paradigm that uses objects and classes to design applications that are modular, reusable and easy to maintain.

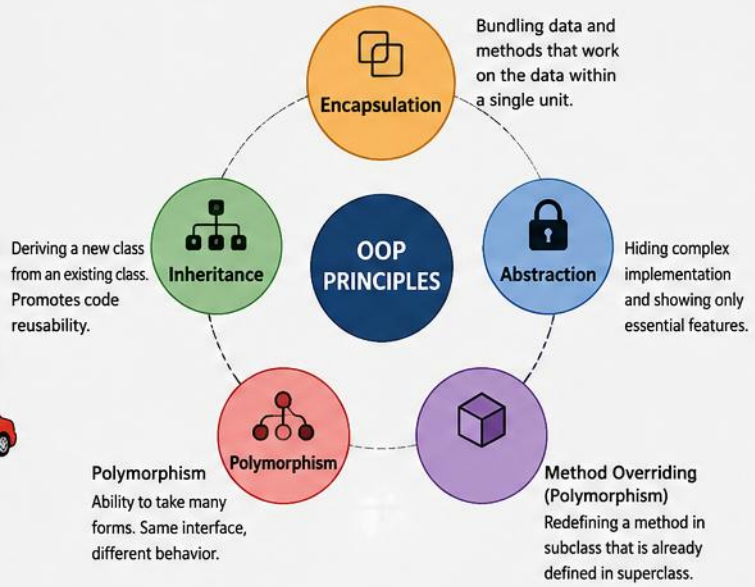
1. CLASS & OBJECT

A Class is a blueprint for creating objects. An Object is an instance of a class.

```
class Car {
  String color;
  int speed;

  void drive() {
    System.out.println("Driving");
  }
}

Car myCar = new Car(); // Object
myCar.color = "Red";
myCar.speed = 120;
```

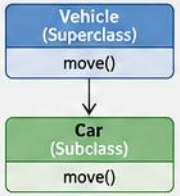


2. KEY OOP PRINCIPLES



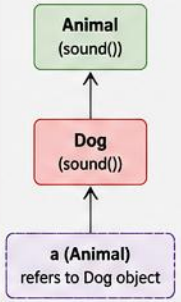
3. INHERITANCE EXAMPLE

```
class Vehicle {
   void move() {
     System.out.println("Moving");
   }
}

class Car extends Vehicle {
   void move() {
     System.out.println("Car is moving");
   }
}

Car c = new Car();
c.move(); // Output: Car is moving
```



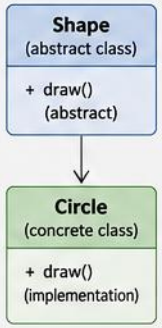
4. POLYMORPHISM EXAMPLE

```
class Animal {
   void sound() {
     System.out.println("Some sound");
   }
}

class Dog extends Animal {
   void sound() {
     System.out.println("Bark");
   }
}

Animal a = new Dog(); // Upcasting
a.sound(); // Output: Bark (runtime polymorphism)
```



5. ABSTRACTION EXAMPLE

```
abstract class Shape {
  abstract void draw(); // abstract method
}

class Circle extends Shape {
  void draw() {
    System.out.println("Drawing Circle");
  }
}

Shape s = new Circle();
s.draw(); // Output: Drawing Circle
```



6. BENEFITS OF OOP

Modularity
Breaks complex programs into smaller, manageable pieces.

Reusability
Code can be reused through inheritance and composition.

Maintainability
Easy to update and modify existing code.

Scalability
Easier to extend and build large applications.

SAMPLE PROGRAM & REAL-WORLD ANALOGY

A first, complete look at classes, constructors, and methods working together.

```
class Person {
  private String name;  private int age;

  public Person(String name, int age) {
    this.name = name;  this.age = age;
  }

  public void introduce() {
    System.out.println("Hi, I am " + name +
      " and I am " + age + " years old.");
  }
  public int getAge() { return age; }
}

public class Main {
  public static void main(String[] args) {
    Person p1 = new Person("Alice", 21);
    p1.introduce();
    System.out.println("Age: " + p1.getAge());
  }
```

🖥️ SAMPLE OUTPUT

```
Hi, I am Alice and I am 21 years old.
Age: 21
```

🚙 REAL-WORLD ANALOGY

- Class: Blueprint of a Car
- Object: A specific Car (e.g., my car)
- Encapsulation: Engine details hidden inside the car
- Inheritance: ElectricCar is a Car
- Polymorphism: Different cars start() differently
- Abstraction: Driver uses steering without knowing internal mechanics

KEY TAKEAWAY Classes define the blueprint; constructors initialize state; methods define behavior.

Why OOP Matters in Enterprise Development

OOP principles help enterprise applications handle complexity, evolve with change, and deliver business value at scale.

1. MANAGE COMPLEXITY

Encapsulation and abstraction break down large systems into manageable classes and modules.

2. IMPROVE MAINTAINABILITY

Well-structured, modular code is easier to understand, test, and maintain over time.

Enterprise Applications

3. ENHANCE REUSABILITY

Classes, interfaces, and components can be reused across projects, reducing time and cost.

4. SUPPORT TEAM COLLABORATION

Encapsulation and clear boundaries allow multiple developers to work in parallel with fewer conflicts.

5. INCREASE RELIABILITY & SECURITY

Data hiding and controlled access reduce unintended side effects and improve system integrity.

6. ENABLE SCALABILITY

OOP design supports the addition of new features and scaling of systems without major rewrites.

BUSINESS IMPACT

- Faster Time to Market
- Lower Total Cost of Ownership
- Higher Quality & Stability
- Adaptable to Change & Future Growth

OOP is not just a coding style—it's a foundation for building robust, maintainable, and future-ready enterprise solutions.

Java

ENTERPRISE

HOW OOP PRINCIPLES DELIVER VALUE IN ENTERPRISE DEVELOPMENT

Six ways OOP pays off in real enterprise development.

Modularity & Reusability

Classes and objects break the system into smaller, reusable components. Reuse saves time and reduces defects. Example: a common Logger or DatabaseConnector class reused across many modules.

Maintainability

Encapsulation hides internal details and localizes changes. Easier to understand, debug, and maintain over time. Example: changing how data is stored doesn't affect other parts of the system.

Extensibility

Inheritance and polymorphism let you extend behavior without modifying existing code. Example: add a new payment method by creating a new class without changing existing code.

Supports Team Collaboration

Clear class responsibilities and interfaces enable multiple developers to work in parallel with minimal conflicts. Example: different teams work on different classes/modules independently.

Scalability

OOP designs are easier to scale as the application grows in size, users, and business complexity. Example: add new features or modules without breaking existing functionality.

Reliability & Security

Encapsulation and abstraction protect critical data and enforce valid usage, reducing errors and security risks. Example: private fields and controlled access ensure data integrity.

Impact on the Business: 🚀 **Faster time to market** 💰 **Lower total cost of ownership** ⭐ **Higher quality software** 🔗 **Easier integration with other systems** 🔄 **Better adaptability to change**

KEY TAKEAWAY OOP is a foundation for building robust, flexible, future-ready applications — it empowers teams to deliver value today and evolve confidently for tomorrow.

LEARNING OBJECTIVES

By the end of this module, you will be able to design, implement, and use object-oriented concepts to build maintainable and extensible Java programs. (1 of 3)

1

Understand Classes and Objects

Define classes as blueprints and create objects as instances with state and behavior.

2

Use Constructors Effectively

Create and use constructors to initialize objects and understand constructor overloading.

3

Apply Encapsulation

Use access modifiers to protect data and provide controlled access through getters and setters.

4

Implement Inheritance

Create new classes by extending existing classes to reuse and extend functionality.

5

Leverage Polymorphism

Understand method overriding and use polymorphism to write flexible and extensible code.

KEY TAKEAWAY In this module, you will build ten core capabilities — see part 2 of 3 for the rest.

LEARNING OBJECTIVES (CONTINUED)

(2 of 3)

6

Achieve Abstraction

Define abstract classes and interfaces to focus on essential behavior and hide implementation details.

7

Understand Relationships Between Classes

Recognize and model relationships: Association, Aggregation, Composition.

8

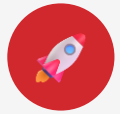
Design Real-World OOP Models

Analyze real-world scenarios and design class diagrams to model entities and their interactions.

9

Write Clean and Maintainable Code

Follow OOP best practices: Naming, Structure, Cohesion, Low coupling.



10

Build OOP Programs

Develop complete Java programs using OOP concepts to solve practical problems.

KEY TAKEAWAY Objectives 7 and 9 cover class relationships and clean-code practices in more depth ahead.

YOU WILL WORK WITH

The building blocks you'll use, the systems you'll practice on, and how you'll know you've succeeded. (3 of 3)

🛠️ YOU WILL WORK WITH

- Classes, objects, and constructors
- Access modifiers and encapsulation
- Inheritance hierarchies
- Interfaces and abstract classes
- Polymorphism and method overriding
- UML class diagrams
- OOP best practices and design principles
- Success check: classes, encapsulation, inheritance
- Success check: polymorphism, UML, full programs

Vehicle Management System

Model a Car system using classes and inheritance:

- Vehicle
- Car
- Truck
- Bike

Banking System

Encapsulate BankAccount data and expose safe operations:

- deposit()
- withdraw()
- getBalance()

Payment System

Use interfaces to support multiple payment types via polymorphism:

- CreditCard
- PayPal
- UPI

KEY TAKEAWAY Example outcomes — Vehicle Management, Banking, and Payment systems — put every concept into practice.

BUSINESS SCENARIO: BANKING MANAGEMENT SYSTEM

We will use a real-world banking scenario to understand and apply OOP concepts in Java — the same scenario this module's graded lab builds into a full system. (1 of 3)

📄 SCENARIO OVERVIEW

A bank wants a system to manage its customers, accounts, and transactions. The system should be:

- Secure
- Easy to maintain
- Extendable to support new types of accounts and services

🏦 MAIN ENTITIES IN THE SYSTEM

Customer

Represents a bank customer. Has personal details and one or more accounts.

Account

Abstract concept for all types of accounts. Has account number, balance, and common operations.

Transaction

Represents a transaction on an account. Stores type, amount, date, and description.

✅ KEY REQUIREMENTS

- Customers can have one or more accounts
- Support Savings, Current, and Fixed Deposit accounts
- Perform deposits, withdrawals, and balance inquiries
- Maintain transaction history for each account
- Ensure data security through encapsulation
- Allow future extensions (e.g., Loan Account)

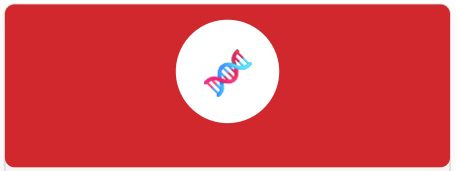
KEY TAKEAWAY A bank needs a system that is secure, easy to maintain, and extendable to new account types.

OOP CONCEPTS APPLIED

Every pillar of OOP maps directly onto the banking scenario. (2 of 3)

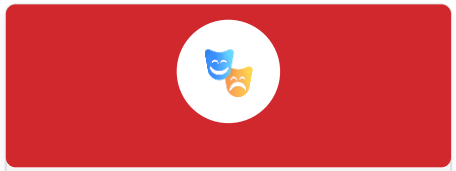
Encapsulation

Protect account details (balance, account number) using private fields and public methods.



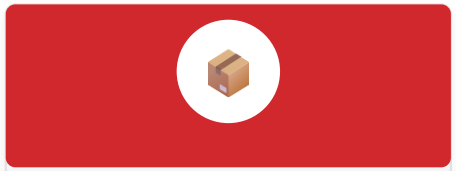
Inheritance

Account is the base class. Derived classes include SavingsAccount, CurrentAccount, FixedDepositAccount.



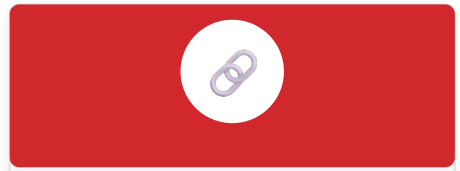
Polymorphism

Different account types override methods such as calculateInterest() and applyCharges().



Abstraction

Define abstract methods in the Account class that must be implemented by subclasses.



Association

Customer has Accounts. Account has Transactions.

KEY TAKEAWAY Encapsulation, Inheritance, Polymorphism, Abstraction, and Association — all five map onto one real system.

EXAMPLE USE CASE FLOW

From onboarding a customer to generating reports. (3 of 3)

1	**Create Customer**	Add a new customer to the bank.
2	**Open Account**	Customer opens a Savings Account.
3	**Perform Transactions**	Deposit or withdraw money from the account.
4	**View Balance & History**	Check current balance and transaction history.
5	**Generate Reports**	Bank generates reports: total deposits, active accounts, and other statistics.

KEY TAKEAWAY Learning Outcome: design and implement a modular, reusable, and extensible banking system using core OOP principles.

WHAT IS OBJECT-ORIENTED PROGRAMMING?

OOP organizes software design around data, or objects, rather than functions and logic alone. (1 of 2)

💡 KEY IDEA

OOP models real-world entities as objects that have:

- State — attributes / data
- Behavior — methods / actions
- Identity — unique reference

🚙 REAL-WORLD ANALOGY — THINK OF A CAR

STATE (DATA)

Make: Toyota
Model: Camry
Color: Red
Speed: 60 km/h

BEHAVIOR (METHODS)

```
start()
accelerate()
brake()
stop()
```

IDENTITY — each car has a unique identity:

- VIN (Vehicle Identification Number)
- Registration Number

Object-Oriented Programming (OOP) is a programming paradigm that organizes software design around data, or objects, rather than functions and logic alone.

KEY TAKEAWAY Every object has State, Behavior, and Identity — the three ingredients OOP is built on.

OBJECT ORIENTED PROGRAMMING (OOP)

A programming paradigm that organizes code around objects and classes rather than functions and logic.

KEY PRINCIPLES OF OOP

1. CLASS & OBJECT
Class is a blueprint.
Object is an instance of a class.

2. ENCAPSULATION
Binding data (fields) and methods together and restricting direct access to some of the object's components.

3. INHERITANCE
Deriving a new class from an existing class to reuse code and extend functionality.

4. POLYMORPHISM
One interface, multiple implementations.
The same method behaves differently for different objects.

5. ABSTRACTION
Hiding complex implementation and showing only essential features to the user.

CLASS VS OBJECT

CLASS (Blueprint)

Car

Attributes (Fields)
- String brand
- String model
- int speed

Methods
+ start()
+ accelerate(int s)
+ stop()

A class defines the structure (attributes) and behavior (methods).

OBJECTS (Instances)

Car car1	Car car2
brand = "Toyota" model = "Corolla" speed = 60	brand = "Honda" model = "Civic" speed = 80
start() accelerate(60) stop()	start() accelerate(60) stop()


Objects are instances of a class with actual values.

REAL WORLD ANALOGY

Class: Car

(Blueprint)

Object 1	Object 2	Object 3
		
Toyota Corolla	Honda Civic	BMW X3

The class defines the common properties and behaviors, while objects have their own data and state.

WHY OOP?

- ✅ Improves code reusability
- ✅ Enhances maintainability
- ✅ Supports scalability
- ✅ Models real world entities
- ✅ Promotes clean and organized code

OOP IN JAVA

THE FOUR PILLARS OF OOP

Every OOP concept in Java builds on these four ideas. (2 of 2)

Encapsulation

Bundling data and methods together in a class and restricting direct access to some components.

EXAMPLE

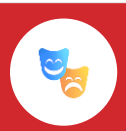
Account balance is hidden (private) and can be accessed only through methods like getBalance().

Inheritance

Creating a new class from an existing class to reuse and extend its functionality.

EXAMPLE

SavingsAccount is-a Account and inherits its properties and methods.



Polymorphism

One interface, multiple implementations. The same method call can behave differently for different objects.

EXAMPLE

calculateInterest() works differently for SavingsAccount and FixedDepositAccount.

Abstraction

Hiding internal implementation details and showing only the essential features.

EXAMPLE

We use an Account object without worrying about how interest is actually calculated.

KEY TAKEAWAY OOP promotes clean design, code reuse, and easier software evolution — essential for robust enterprise applications.

Classes and Objects in Java

A **Class** is a blueprint or template that defines the properties (fields) and behaviors (methods) of objects.
An **Object** is an instance of a class.

1. Class (Blueprint)

```
class Car {
   // Fields (attributes)
   String brand;
   String color;
   int year;

   // Method (behavior)
   void start() {
      System.out.println("Car started");
   }
}
```

Objects are created from a class using the 'new' keyword.

2. Objects (Instances)

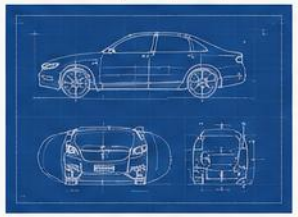
car1	car2	car3
		
brand = "Toyota" color = "Red" year = 2020	brand = "Honda" color = "Blue" year = 2019	brand = "BMW" color = "White" year = 2021
start() (behavior)	start() (behavior)	start() (behavior)

3. Creating Objects in Java

```
// Creating objects
Car car1  = new Car();
Car car2  = new Car();
Car car3  = new Car();

// Setting values
car1.brand = "Toyota";   car1.color = "Red";    car1.year = 2020;
car2.brand = "Honda";    car2.color = "Blue";   car2.year = 2019;
car3.brand = "BMW";      car3.color = "White";  car3.year = 2021;

// Calling method
car1.start();    // Output: Car started
```

4. Real-World Analogy

Class (Blueprint)



Car Design

Objects (Actual Cars)

Different instances with same design but different values

Key Takeaways

- Class defines what objects look like and what they can do.
- Objects are real entities created from a class.
- Multiple objects can be created from the same class, each with its own values.

Java

Write Once, Run Anywhere

EXAMPLE IN JAVA

1. Class Definition (Blueprint) — attributes and methods live together in one class. (1 of 2)

```
class Car {

  int speed;
  String color;
  String model;

  // Method (behavior)
  void start() {
    System.out.println("Car is starting...");
  }

  void stop() {
    System.out.println("Car is stopping...");
  }
}
```

ATTRIBUTES (STATE)

- speed
- color
- model

METHODS (BEHAVIOR)

- start()
- stop()

KEY TAKEAWAY A class bundles attributes (what an object has) and methods (what an object can do) into one blueprint.

EXAMPLE IN JAVA

2. Creating Objects — each object gets its own state, stored separately in memory. (2 of 2)

```java
public class Main {
  public static void main(String[] args) {

    // Creating objects of class Car
    Car myCar = new Car();
    Car yourCar = new Car();
    Car officeCar = new Car();

    // Setting state
    myCar.speed = 120;
    myCar.color = "Red";
    myCar.model = "Toyota Camry";

    // Calling methods
    myCar.start();
    myCar.stop();
  }
}
```

🧠 MEMORY (RUNTIME)

myCar
Speed: 120, Color: Red, Model: Toyota Camry

yourCar
Speed: 80, Color: Blue, Model: Honda City

officeCar
Speed: 100, Color: White, Model: BMW 5 Series

✅ KEY POINTS

- A class is a blueprint; an object is an instance of that class
- Multiple objects can be created from a single class
- Each object has its own state, stored in memory
- Methods define the behavior that objects can perform

KEY TAKEAWAY Each object has its own copy of data, stored in separate memory.

TRY IT YOURSELF — EXERCISE 1: IDENTIFY ENTITIES

Reinforces: turning a real-world domain into classes and objects, as just covered.

1

Goal

List entities, attributes, and one responsibility each from a banking domain

2

Deliverable

A short entity table in notes.md — no code required

3

Domain

Customer, Account, Transaction — the entities from this module's scenario

4

Key concept

Each entity becomes a class; attributes become fields; behavior becomes methods

5

Reference

exercises/exercise-01-domain-entities.md

TRY IT NOW Complete Exercise 1 before continuing — see exercises/exercise-01-domain-entities.md.

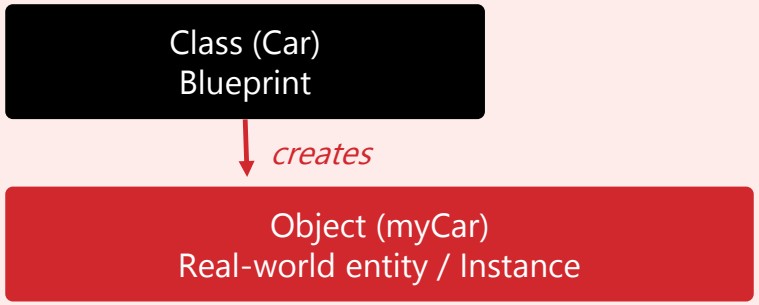
CREATING AND USING OBJECTS

Objects are created from classes and used to access data and call methods. (1 of 2)

❓ 1. WHAT DOES IT MEAN?

An object is an instance of a class. You create objects to represent real-world entities and use them to work with data and behavior.

🔗 RELATIONSHIP



⚙️ 2. HOW TO CREATE AN OBJECT IN JAVA?

SYNTAX

```
ClassName objectName = new ClassName();
```

NOTES

- new keyword allocates memory for the object
- Constructor is called to initialize the object
- objectName is a reference to the object

KEY TAKEAWAY An object is an instance of a class — the new keyword allocates memory and calls the constructor.

CREATING AND USING OBJECTS

3. Example: Car Class and Objects, and the resulting output. (2 of 2)

```
class Car {
  String brand;  int speed;

  void displayInfo() {
    System.out.println("Brand: " + brand +
      ", Speed: " + speed + " km/h");
  }
}

public class Main {
  public static void main(String[] args) {
    Car car1 = new Car();
    Car car2 = new Car();
    car1.brand = "Toyota"; car1.speed = 120;
    car2.brand = "Honda";  car2.speed = 80;
    car1.displayInfo();
    car2.displayInfo();
  }
}
```

🖥️ 5. OUTPUT

```
Brand: Toyota, Speed: 120 km/h
Brand: Honda, Speed: 80 km/h
```

✅ BEST PRACTICES

- Use meaningful names for classes and objects
- Initialize object state properly
- Encapsulate data (private fields, public methods)
- Keep objects focused on a single responsibility

KEY TAKEAWAY Two objects, one class — each car1 and car2 holds its own brand and speed.

Constructors in Java

A constructor is a special method used to initialize objects.
It is called automatically when an object is created.

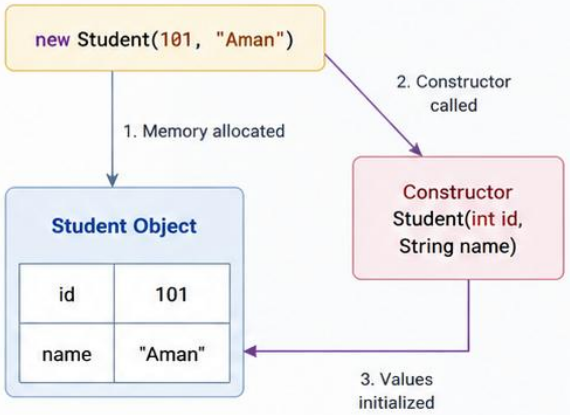
Key Points

- The name of the constructor is the same as the class name.
- It has **no return type**, not even void.
- It is called automatically when an object is created using new.
- Used to initialize object state.
- Constructors can be overloaded.
- If no constructor is defined, Java provides a **default no-arg constructor**.

Example

```
class Student {
    int id;
    String name;

    // Constructor
    Student(int id, String name) {
        this.id = id;
        this.name = name;
    }
}

public class Main {
    public static void main(String[] args) {
      Student s1 = new Student(101, "Aman");
      System.out.println(s1.id + " " + s1.name);
    }
}
```

How It Works



Types of Constructors

1. Default (No-Arg) Constructor

Takes no parameters.
Provided by Java if no constructor is defined.

```
class Student {
  int id;
  String name;

   // Default constructor
   Student() {
      id = 0;
      name = "Unknown";
   }
}
```

Usage:
`Student s = new Student();`

2. Parameterized Constructor

Takes parameters to initialize object with specific values.

```
class Student {
  int id;
  String name;

   // Parameterized constructor
   Student(int id, String name) {
      this.id = id;
      this.name = name;
   }
}
```

Usage:
`Student s = new Student(101, "Aman");`

Rules & Best Practices

- Constructor name must match the class name (exact case).
- Constructor has no return type.
- Use constructors to ensure objects are properly initialized.
- Use constructor overloading for flexibility.
- Keep constructors simple and focused on initialization.
- Avoid heavy logic in constructors.

Summary

Constructors ensure that an object is in a valid state when it is created.
They play a vital role in initializing and maintaining object data.

new Keyword → Constructor Called → Object Initialized → Object Ready to Use

CONSTRUCTORS IN JAVA

4. Example — the Car class with both constructors. 5. How Constructors Work. (1 of 3)

```
class Car {
  String brand;
  int speed;

  // Default constructor
  Car() {
    brand = "Unknown";
    speed = 0;
  }

  // Parameterized constructor
  Car(String brand, int speed) {
    this.brand = brand;
    this.speed = speed;
  }

  void display() {
    System.out.println("Brand: " + brand +
      ", Speed: " + speed + " km/h");
  }
```

🔄 5. HOW CONSTRUCTORS WORK

1. Object created using new — memory is allocated
2. The appropriate constructor is called
3. Constructor initializes the member variables
4. The object is ready to use
5. Methods can be called on the object

MEMORY (HEAP) — Car Object

```
brand → "Toyota"        speed → 120
```

KEY TAKEAWAY new allocates memory, the constructor runs, fields are initialized, then the object is ready to use.

CONSTRUCTORS IN JAVA

6. Example Output. 7. Constructor Overloading — multiple constructors, different parameter lists. (2 of 3)

```
Car car1 = new Car();
Car car2 = new Car("Toyota", 120);

car1.display();
car2.display();
```

OUTPUT

```
Brand: Unknown, Speed: 0 km/h
Brand: Toyota, Speed: 120 km/h
```

```
class Student {
  String name;
  int age;
  double cgpa;

  Student() {
    name = "Unknown"; age = 0; cgpa = 0.0;
  }
  Student(String name) {
    this.name = name;
  }
  Student(String name, int age) {
    this.name = name; this.age = age;
  }
  Student(String name, int age, double cgpa) {
    this.name = name; this.age = age;
    this.cgpa = cgpa;
  }
}
```

KEY TAKEAWAY A class can have multiple constructors with different parameter lists — constructor overloading.

CONSTRUCTORS IN JAVA

8. Rules for Constructors. 9. this() — Calling Another Constructor. (3 of 3)

📋 8. RULES FOR CONSTRUCTORS

- Constructor name must be the same as the class name
- Constructor must not have a return type
- Constructors are called automatically
- If no constructor is provided, Java creates a default constructor
- this() can be used to call another constructor in the same class

🏆 KEY TAKEAWAY

Constructors ensure objects start with a valid and consistent state. Constructor overloading provides flexibility.

➡️ 9. THIS() — CALLING ANOTHER CONSTRUCTOR

```
class Car {
  String brand;  int speed;
  Car() {
    this("Unknown", 0);
  }
  Car(String brand, int speed) {
    this.brand = brand;  this.speed = speed;
  }
}
```

this() must be the first statement in the constructor body.

- Constructors initialize objects when they are created
- Use parameterized constructors to pass values while creating objects

KEY TAKEAWAY Constructors initialize objects; this() avoids duplicated initialization logic across overloaded constructors.

ENCAPSULATION IN JAVA

Encapaulation is the OOP principle of wrapping data (variables) and code (methods) together as a single unit and restricting direct access to some of an object's components.

BENEFITS

Data Hiding
Protects data from unauthorized access.

Data Integrity
Validates and controls data through methods.

Maintainability
Easier to modify internal implementation.

Reusability
Promotes clean and reusable code.

Class: BankAccount

Private Data (fields)

```
private double balance;
private String accountNumber;
private String ownerName;
```

Not accessible directly outside the class

Public Methods

```
public void deposit(double amount)
public void withdraw(double amount)
public double getBalance()
public String getAccountNumber()
```

Accessible from outside the class

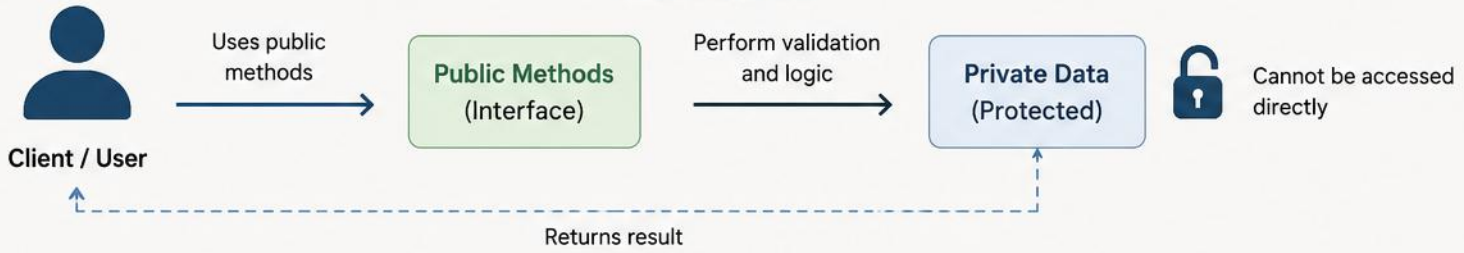
WHAT IS HIDDEN?

The internal state (fields/variables) is hidden using the **private** access modifier.

HOW IS ACCESS PROVIDED?

Access is provided through **public** methods called getter and setter (or other methods).

HOW IT WORKS



UNDERSTANDING ENCAPSULATION

3. Real-World Analogy. 4. Example in Java — the BankAccount class.

🏦 3. REAL-WORLD ANALOGY — BANK EXAMPLE

You (Customer)

Bank Vault (Private Data)

You cannot access the vault directly. Instead, you interact through bank staff (Public Methods) to:

- Deposit money
- Withdraw money
- Check account balance
- Protects data; validates every change
- Keeps internals flexible to maintain

```
public class BankAccount {
  private String accountNumber;
  private double balance;

  public double getBalance() { return balance; }

  public void deposit(double amount) {
    if (amount > 0) { balance += amount; }
  }

  public void withdraw(double amount) {
    if (amount > 0 && amount <= balance) {
      balance -= amount;
    }
  }
}
```

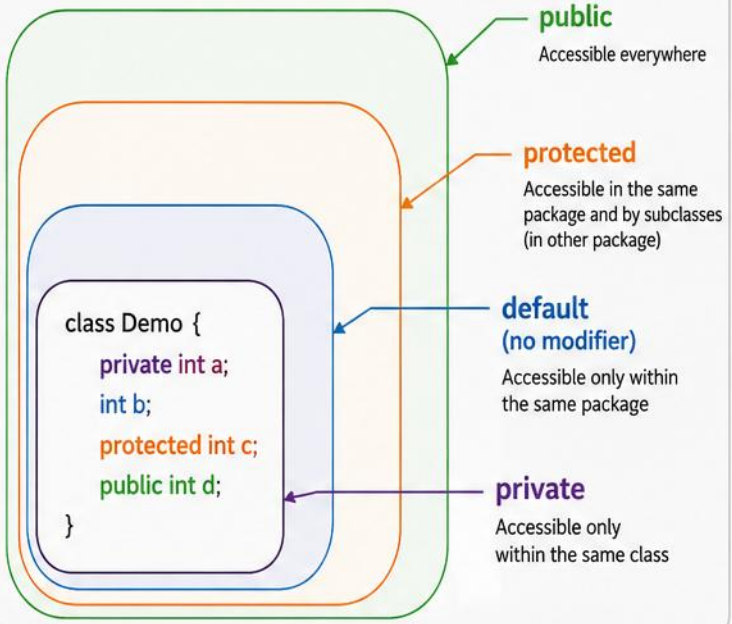
KEY TAKEAWAY You interact with a bank vault through staff, never directly — the same idea protects an object's data. Remember: Encapsulation = Data Hiding + Controlled Access.

Access Modifiers in Java

Access modifiers control the visibility (scope) of classes, methods, and variables.

Modifier	Within Same Class	Within Same Package	Subclasses in Different Package	Outside Package
private	✓	✗	✗	✗
default (no modifier)	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Where Can They Be Accessed?



private

- Accessible only within the same class.
- Not visible outside the class.

default (no modifier)

- Accessible within the same package.
- Not visible in other packages.

protected

- Accessible within the same package.
- Also accessible in subclasses even if they are in a different package.

public

- Accessible from anywhere.
- No restrictions.

Java

ACCESS MODIFIERS

ACCESS MODIFIERS IN JAVA

3. Key Points. 4. Example — all four modifiers used in one class. (1 of 2)

📌 3. KEY POINTS

- Top-level classes can be public or default (no modifier) only
- Member variables, methods, and constructors can use any of the four access modifiers
- If no modifier is specified, it is default by default
- More restrictive access is generally safer and encourages encapsulation
- Public is the most permissive; private is the most restrictive

```
package com.bank.account;
public class Account {
  private String accountNumber; // private
  double balance;              // default
  protected double interestRate; // protected
  public String bankName;   // public

  // Constructor
  public Account(String accountNumber,
      double balance) {
    this.accountNumber = accountNumber;
    this.balance = balance;
    this.interestRate = 4.5;
    this.bankName = "Trusted Bank";
  }
}
```

KEY TAKEAWAY One class, four modifiers — each field's visibility is chosen deliberately.

ACCESS MODIFIERS IN JAVA

5. Example Scenario. 6. Real-World Analogy. (2 of 2)

```java
// SavingsAccount is in com.bank.savings
// It extends Account, in com.bank.account
package com.bank.savings;
import com.bank.account.Account;

public class SavingsAccount extends Account {
  public void showInfo() {
    // accountNumber -> Not accessible (private)
    // balance -> Not accessible (default)
    // interestRate -> Accessible (protected)
    System.out.println(interestRate); // OK
    // bankName -> Accessible (public)
    System.out.println(bankName);     // OK
    // System.out.println(accountNumber); // Error
  }
}
```

🏢 6. REAL-WORLD ANALOGY

🔐 `private`	Manager's safe — only the manager can access it.
🚪 `default`	Office rooms inside the bank — staff in the same office can access.
🏢 `protected`	Departments and authorized branches — accessible to related departments.
🌍 `public`	Public information — available to customers and everyone.

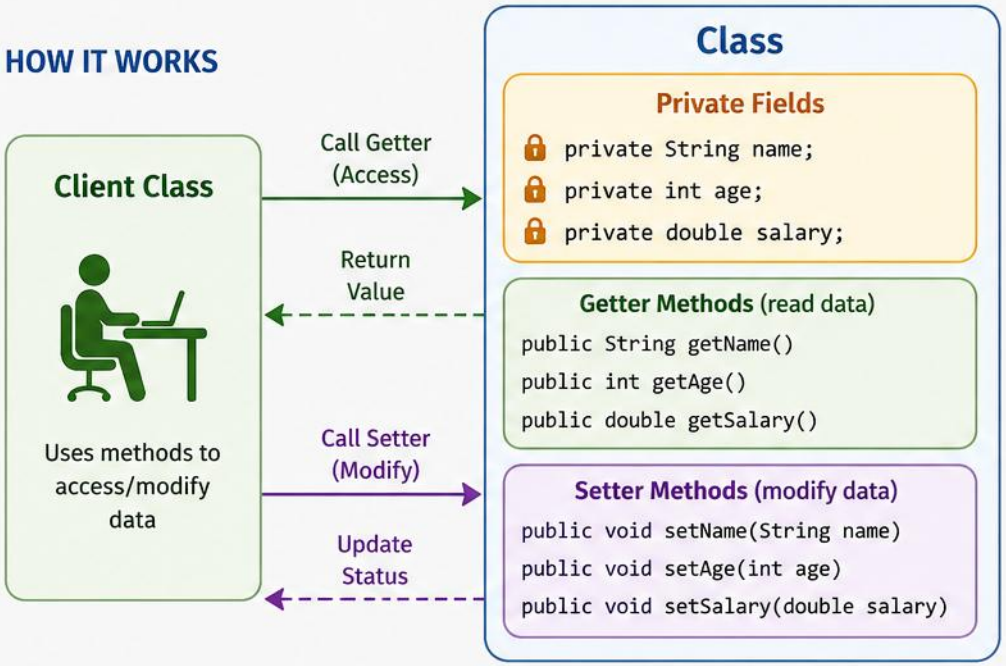
⭐ **BEST PRACTICE** Use the most restrictive access level that still allows the class to function correctly.

KEY TAKEAWAY Access modifiers are fundamental to OOP — they enforce encapsulation, protect data, and build secure, maintainable applications.

GETTERS AND SETTERS IN JAVA

Getters and setters are methods used to access and modify the private fields (attributes) of a class. Theey provide controlled access and help in encapsulation.

HOW IT WORKS



EXAMPLE

```
public class Employee {
   private String name;
   private int age;
   private double salary;

   // Getter methods
   public String getName() { return name; }
   public int getAge() { return age; }
   public double getSalary() { return salary; }

   // Setter methods
   public void setName(String name) { this.name = name; }
   public void setAge(int age) { this.age = age; }
   public void setSalary(double salary) { this.salary =
            salary; }

}
```

BENEFITS

Encapsulation
Protects data by controlling access.

Data Validation
Add checks in setters before updating.

Flexibility
Change internal implementation without affecting client code.

Read-Only Access
Provide only getter if data should not be modified.

BEST PRACTICES

- Keep fields private.
- Use meaningful method names: getX(), setX().
- Validate data in setter methods.
- Avoid placing business logic in getters and setters.

GETTERS AND SETTERS IN JAVA

3. Example: BankAccount Class — getters and setters with validation.

```
public class BankAccount {
  private String accountNumber;   private double balance;
  public BankAccount(String accountNumber, double balance) {
    this.accountNumber = accountNumber;  this.balance = balance;
  }
  public String getAccountNumber() { return accountNumber; }
  public void setAccountNumber(String accountNumber) {
    if (accountNumber != null && !accountNumber.isBlank()) {
      this.accountNumber = accountNumber;
    } else { System.out.println("Invalid account number!"); }
  }
  public double getBalance() { return balance; }
  public void setBalance(double balance) {
    if (balance >= 0) { this.balance = balance; }
    else { System.out.println("Balance cannot be negative!"); }
  }
}   // Variations: read-only (getter only), write-only (setter only), computed (e.g. getAnnualInterest())
```

KEY TAKEAWAY Setters validate before assigning — an empty account number or negative balance is rejected. Variations: read-only, write-only, and computed getters/setters.

TRY IT YOURSELF — EXERCISE 2: ENCAPSULATION PRACTICE

Reinforces: private fields, access modifiers, and getters/setters you just saw.

1

Goal

Create Account with a private balance and deposit / withdraw methods

2

File

Account.java in examples/module-03-exercises/

3

Key concept

private double balance — no public field writes, all changes go through methods

4

Compile & Run

javac Account.java EncapsulationDemo.java && java EncapsulationDemo

5

Reference

exercises/exercise-02-encapsulation.md

TRY IT NOW Complete Exercise 2 before continuing — see exercises/exercise-02-encapsulation.md.

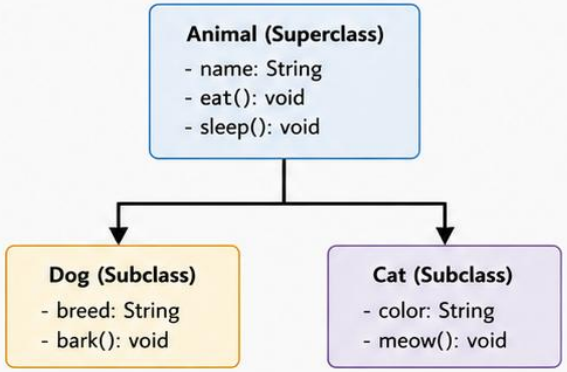
INHERITANCE IN JAVA

Inheritance is a mechanism in Java where one class acquires the properties (fields and methods) of another class.

KEY POINTS

- Inheritance promotes code reusability and establishes a relationship between classes.
- The class whose members are inherited is called the Superclass (Parent).
- The class that inherits is called the Subclass (Child).
- In Java, a class can extend only one class (Single Inheritance).
- Use the `extends` keyword to inherit a class.

CLASS HIERARCHY (EXAMPLE)

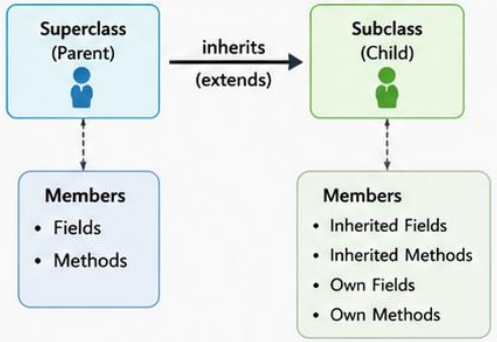


Dog and Cat inherit name, eat(), and sleep() from `Animal`.

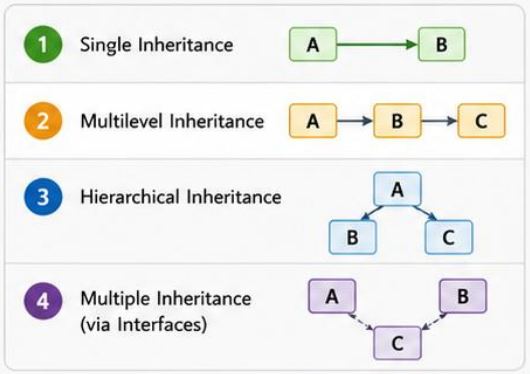
EXAMPLE CODE

```
// Superclass
class Animal {
    String name;
    void eat() {
        System.out.println("Eating...");
    }
    void sleep() {
        System.out.println("Sleeping...");
    }
}
// Subclass
class Dog extends Animal {
    String breed;
    void bark() {
       System.out.println("Barking...");
    }
}
```

HOW INHERITANCE WORKS



TYPE OF INHERITANCE IN JAVA



BENEFITS

Code Reusability
Reusing fields and methods of existing classes.

Extensibility
Allows adding new features in the subclass.

Maintainability
Improves code organization and readability.

Polymorphism Support
Enables method overriding and dynamic binding.

Inheritance represents an IS-A relationship.
Example: `Dog IS-A Animal`, `Cat IS-A Animal`.

Use `extends` keyword to create a subclass.
`class SubClass extends SuperClass { ... }`

Java

UNDERSTANDING INHERITANCE

3. Syntax in Java. 4. Example — using an inherited Car object. (1 of 3)

```
// Superclass
class Vehicle {
  String brand;
  int maxSpeed;

  void start() {
    System.out.println("Vehicle started");
  }
  void stop() {
    System.out.println("Vehicle stopped");
  }
}

// Subclass
class Car extends Vehicle {
  int numDoors;
  void playMusic() {
    System.out.println("Playing music");
  }
}
```

```
public class TestInheritance {
  public static void main(String[] args) {
    Car myCar = new Car();
    myCar.start(); myCar.stop();      // inherited
    myCar.numDoors = 4;               // own
    myCar.playMusic();
    myCar.brand = "Toyota";           // inherited
    myCar.maxSpeed = 180;
    System.out.println("Brand: " + myCar.brand);
    System.out.println("Max Speed: " + myCar.maxSpeed);
  }
}
```

```
OUTPUT:  Vehicle started → Vehicle stopped → Playing music
→ Brand: Toyota → Max Speed: 180
```

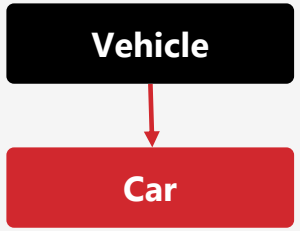
KEY TAKEAWAY myCar can use both its own members and everything it inherited from Vehicle.

UNDERSTANDING INHERITANCE

5. Types of Inheritance in Java — three structural forms. (2 of 3)

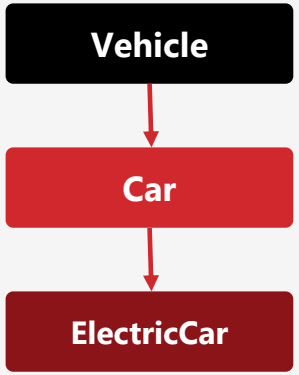
SINGLE

A subclass inherits from exactly one superclass.



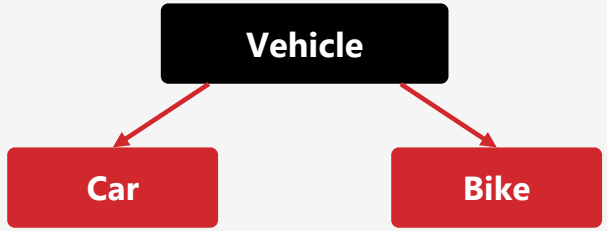
MULTILEVEL

A subclass inherits from a class that is itself a subclass.



HIERARCHICAL

Multiple subclasses inherit from the same superclass.



💡 NOTE — METHOD OVERRIDING

A subclass can provide its own implementation of a method already defined in its superclass — covered in detail on the next slide.

KEY TAKEAWAY Java supports single, multilevel, and hierarchical class inheritance — but not multiple class inheritance.

UNDERSTANDING INHERITANCE

6. Method Overriding. 7. super Keyword. 8. Key Takeaways. (3 of 3)

```java
class Vehicle {
  void start() {
    System.out.println("Vehicle started");
  }
}
class Car extends Vehicle {
  @Override
  void start() {
    System.out.println("Car started with key");
  }
}
```

📝 6. METHOD OVERRIDING — NOTE

The overridden method must have the same method name, the same parameter list, and the same (or compatible) return type.

```java
class Vehicle {
  String brand;
  Vehicle(String brand) { this.brand = brand; }
}
class Car extends Vehicle {
  int numDoors;
  Car(String brand, int numDoors) {
    super(brand);   // calls parent ctor
    this.numDoors = numDoors;
  }
}
```

7. super KEYWORD — accesses superclass members and calls the superclass constructor.

🏆 8. KEY TAKEAWAYS

- Inheritance models the IS-A relationship; use extends to inherit
- A subclass gets all accessible members of the superclass
- Method overriding allows specialized implementations
- Use super to access parent members and constructors

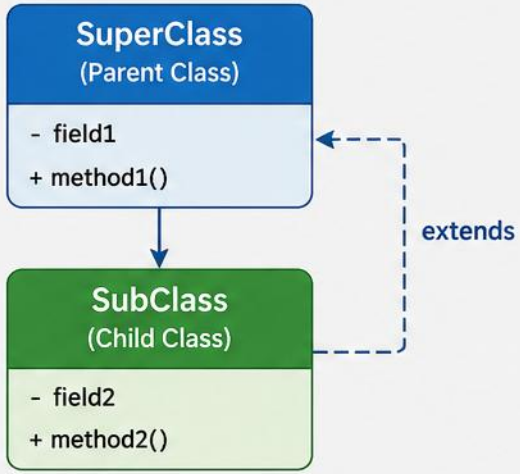
Goal: use inheritance wisely to model real-world relationships and write better OOP Java programs

KEY TAKEAWAY Good Practice: design inheritance only when there's a clear IS-A relationship. Remember: it leads to clean, reusable, extensible code.

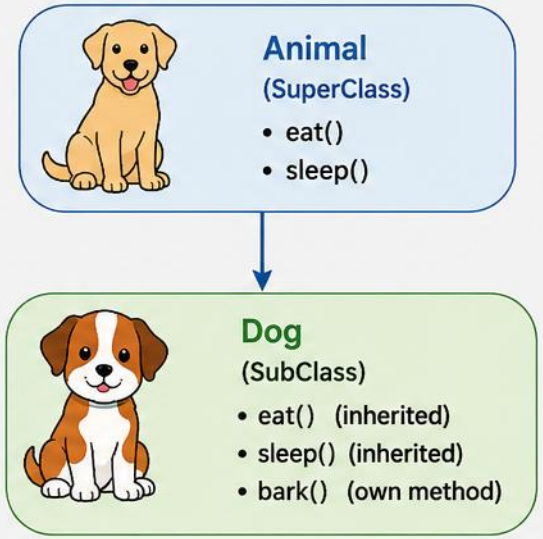
SINGLE INHERITANCE IN JAVA

Single inheritance is a feature in Java where a subclass inherits properties and behaviors from a single superclass.

CLASS DIAGRAM



VISUAL REPRESENTATION



CODE EXAMPLE

```
// Parent Class
class Animal {
    void eat() {
        System.out.println("Animal is eating");
    }
}
// Child Class
class Dog extends Animal {
    void bark() {
        System.out.println("Dog is barking");
    }
}
```

KEY POINTS

- A subclass inherits fields and methods from one superclass only.
- Use the `extends` keyword to achieve single inheritance.
- The subclass can add its own fields and methods.
- Promotes code reusability and establishes an IS-A relationship.

SINGLE INHERITANCE IN JAVA

3. Example — the Animal superclass and Dog subclass. (1 of 3)

```java
// Superclass: Animal.java
class Animal {

  String name;
  int age;

  void eat() {
    System.out.println(name + " is eating");
  }

  void sleep() {
    System.out.println(name + " is sleeping");
  }
}
```

```java
// Subclass: Dog.java
class Dog extends Animal {

  String breed;

  void bark() {
    System.out.println(name + " is barking");
  }
}
```

KEY TAKEAWAY Dog reuses Animal's name, age, eat(), and sleep() — and adds breed and bark() of its own. (Private members are never inherited — only public/protected members are.)

SINGLE INHERITANCE IN JAVA

4. Usage — TestInheritance.java puts both classes to work. (2 of 3)

```java
public class TestInheritance {
  public static void main(String[] args) {

    Dog d = new Dog();

    // Inherited fields
    d.name = "Tommy";
    d.age = 3;

    // Own field
    d.breed = "Labrador";

    // Inherited methods
    d.eat();      // from Animal
    d.sleep();    // from Animal

    // Own method
    d.bark();     // from Dog
  }
}
```

OUTPUT

```
Tommy is eating
Tommy is sleeping
Tommy is barking
```

One Dog object reaches all three methods — two inherited from Animal, one its own.

KEY TAKEAWAY Inherited fields and methods behave exactly like the subclass's own — no extra syntax needed to use them.

SINGLE INHERITANCE IN JAVA

7. super Keyword Example. 8. Summary. (3 of 3)

```
class Animal {
  String name = "Animal";
  Animal() {
    System.out.println("Animal constructor");
  }
  void show() {
    System.out.println("Animal show()");
  }
}

class Dog extends Animal {
  String name = "Dog";
  Dog() {
    super();   // calls Animal constructor
    System.out.println("Dog constructor");
  }
  void show() {
    super.show();   // calls Animal show()
    System.out.println("Dog show()");
  }
```

🏁 8. SUMMARY

- Models a real-world IS-A relationship (e.g., Dog IS-A Animal)
- Promotes code reusability and reduces duplication
- Helps build scalable and maintainable applications
- Forms the foundation for multilevel and hierarchical inheritance

💡 REMEMBER

Single inheritance means "one parent, many children." Java does not support multiple inheritance with classes, but it does support it through interfaces.

KEY TAKEAWAY Single inheritance models real-world IS-A relationships and reduces code duplication.

METHOD OVERRIDING IN JAVA

A subclass provides a specific implementation of a method already defined in its superclass.

📋 RULES FOR OVERRIDING

- Same method name and parameters
- Same or covariant return type
- Access level cannot be more restrictive
- static, final, private methods cannot be overridden
- Use @Override to catch mistakes

```
class Animal {
  public void sound() {
    System.out.println("...");
  }
}

class Dog extends Animal {
  @Override
  public void sound() {
    System.out.println("Bark");
  }
}
```

⛔ WHAT CANNOT BE OVERRIDDEN?

- final methods
- static methods (hidden, not overridden)
- private methods
- constructors

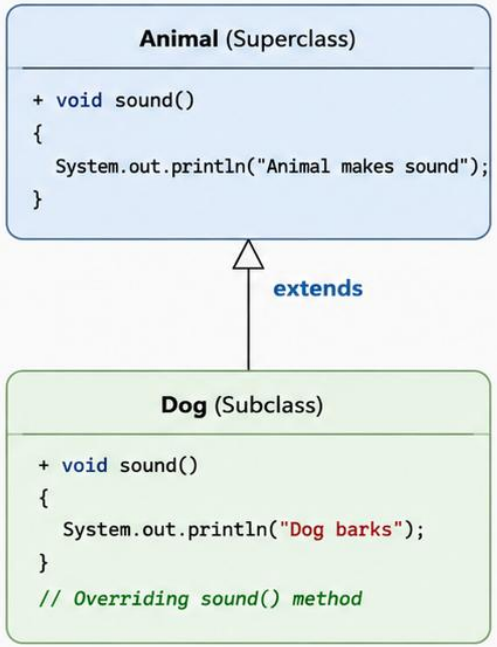
Overriding vs Overloading: overriding requires inheritance and runtime binding; overloading is same-class, compile-time binding.

KEY TAKEAWAY Method overriding is the key that unlocks runtime polymorphism in Java.

Method Overriding in Java

Method overriding allows a subclass to provide a specific implementation of a method that is already defined in its superclass.

Class Hierarchy

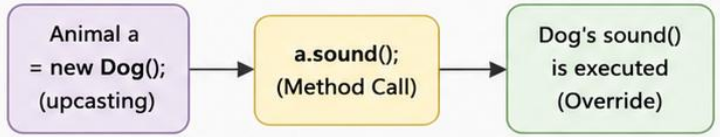


Code Example

```java
class Animal {
    void sound() {
        System.out.println("Animal makes sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Dog barks");
    }
}

public class Test {
    public static void main(String[] args) {
        Animal a = new Dog();   // upcasting
        a.sound();              // calls Dog's sound()
    }
}
```

Subclass provides its own implementation

How It Works (Runtime)



The actual method executed is decided at **runtime** based on the object (**Dog**), not the reference type (**Animal**).

Key Points

- Method name, parameters and return type must be same.
- Access level in subclass must be same or broader (cannot be more restrictive).
- @Override annotation is optional but recommended.
- Method overriding supports runtime polymorphism.
- It is used to provide specific behavior in the subclass.

Benefit: Achieves runtime polymorphism and promotes code reusability and extensibility.

TRY IT YOURSELF — EXERCISE 3: INHERITANCE SKETCH

Reinforces: extends and method overriding, as just covered.

1

Goal

Create SavingsAccount and CurrentAccount extending Account

2

File

SavingsAccount.java, CurrentAccount.java in examples/module-03-exercises/

3

Key concept

extends Account, then override withdraw() or getAccountType()

4

Compile & Run

javac *.java && java InheritanceDemo — call both types via an Account reference

5

Reference

exercises/exercise-03-inheritance.md

TRY IT NOW Complete Exercise 3 before continuing — see exercises/exercise-03-inheritance.md.

COMPILE-TIME POLYMORPHISM

(Method Overloading in Java)

Compile-time polymorphism is achieved by method overloading –
same method name with different parameter lists (number, type or order of parameters).

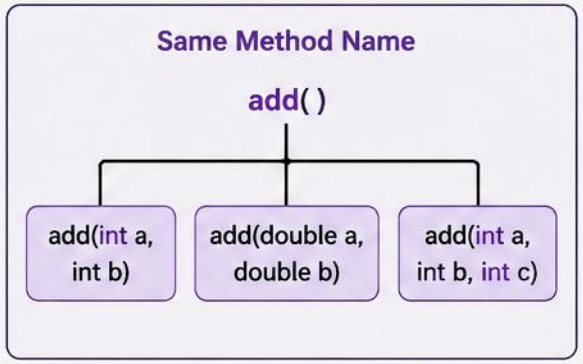
HOW IT WORKS

1. Multiple methods with the same name but different parameters
2. Compiler checks the method signature at compile time
3. Appropriate method is selected based on arguments passed
4. Method is bound at compile time

EXAMPLE

```
class Calculator {

    int add(int a, int b) {
        return a + b;
    }

    double add(double a, double b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}

public class Demo {
   public static void main(String[] args) {
     Calculator c = new Calculator();

     System.out.println(c.add(10, 20));      // calls add(int, int)
     System.out.println(c.add(10.5, 20.5)); // calls add(double, double)
     System.out.println(c.add(10, 20, 30)); // calls add(int, int, int)
   }
}
```

METHOD OVERLOADING



KEY POINTS

- Occurs within the same class.
- Method signature must be different.
- Return type can be same or different (not part of signature).
- Achieved at compile time.
- Improves code readability and reusability.

OUTPUT

```
30
31.0
60
```

RUNTIME POLYMORPHISM IN JAVA

— Achieved through Method Overriding —

Runtime polymorphism allows a method call to be resolved at runtime.
The same method call behaves differently for different objects.

Superclass Animal

```
void sound() {
  System.out.println("Animal sound");
}
```

Subclass Dog

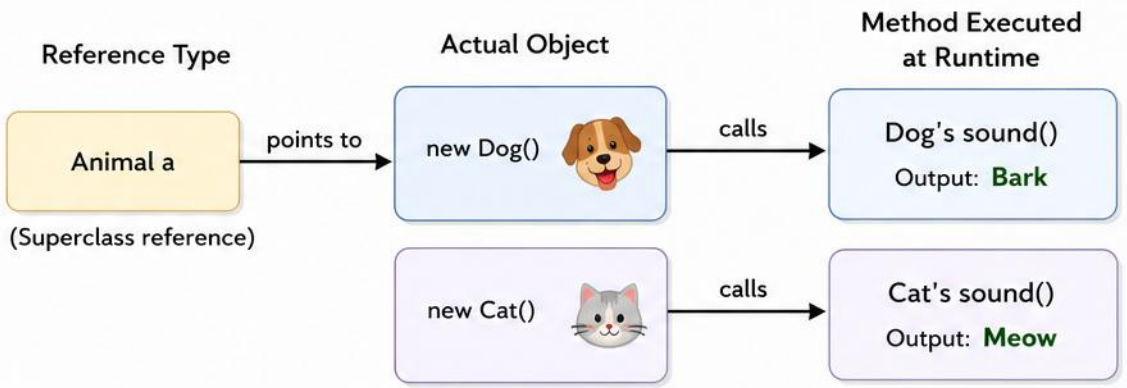
```
@Override
void sound() {
  System.out.println("Bark");
}
```

Subclass Cat

```
@Override
void sound() {
  System.out.println("Meow");
}
```

Class Hierarchy

How it works at runtime (Dynamic Method Dispatch)



Example Code

```
Animal a;

a = new Dog();
a.sound();  // Output: Bark

a = new Cat();
a.sound();  // Output: Meow
```

Explanation

The reference variable 'a' is of type **Animal**, but it can point to objects of Dog or Cat.

The **JVM** determines at **runtime** which object the reference points to and calls the overridden method of that object.

Key Points

- Achieved through method overriding
- Decision is made at **runtime**
- Requires inheritance
- Provides flexibility and extensibility

Resolved at Runtime

Runtime Polymorphism = One interface, multiple implementations, **decision at runtime**.

COMPILE-TIME VS RUNTIME POLYMORPHISM

Java achieves polymorphism two ways: overloading and overriding.

ASPECT	COMPILE-TIME (Overloading)	RUNTIME (Overriding)
Binding	Early binding	Late binding
Occurs in	Same class	Parent & child classes
Signature	Must differ	Must be identical
Decided by	Compiler	JVM at runtime
Performance	Faster	Slightly slower

`Overloading: int add(int,int) vs double add(double,double)`

`Overriding: Animal a = new Dog(); a.sound(); runs Dog.sound()`

KEY TAKEAWAY Overloading is resolved by the compiler on the method signature; overriding is resolved by the JVM on the actual object.

DYNAMIC METHOD DISPATCH IN JAVA

How an overridden method call is resolved at runtime, based on the actual object type.

Animal a = new Dog(); → **a.sound() called** → **JVM checks actual type** → **Executes Dog.sound()**

🧠 MEMORY VIEW

```
Stack (reference)          Heap (object)
a  →  0x101                  Dog object
                                    { sound() →
Dog.sound() }
```

Reference 'a' is typed Animal, but points to a Dog object — so Dog.sound() runs.

❓ WHY IT'S CALLED DYNAMIC

- The method call isn't decided until runtime
- The JVM inspects the actual object type
- Requires inheritance and method overriding
- Private, static, final methods are not dispatched dynamically

KEY TAKEAWAY Dynamic dispatch is runtime polymorphism, powered by inheritance and method overriding.

Dynamic Method Dispatch in Java

The JVM determines which overridden method to call at runtime based on the actual object type, not the reference type.

class Animal

```
void sound() {
    System.out.println("Animal makes sound");
}
}
```

extends

class Dog extends Animal

```
@Override
void sound() {
    System.out.println("Dog barks");
}
}
```

extends

class Cat extends Animal

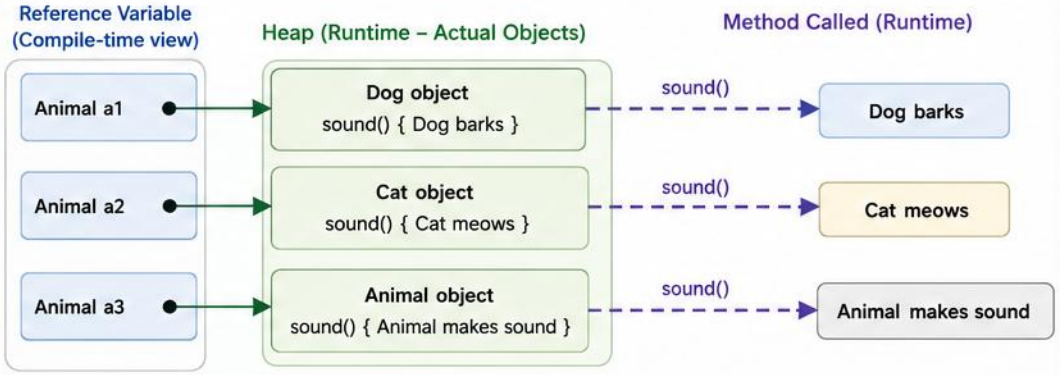
```
@Override
void sound() {
    System.out.println("Cat meows");
}
}
```

```
public class Demo {
   public static void main(String[] args) {
      Animal a1 = new Dog();
      a1.sound();              // Dog barks

      Animal a2 = new Cat();
      a2.sound();              // Cat meows

      Animal a3 = new Animal();
      a3.sound();              // Animal makes sound
   }
}
```

What happens at runtime?

1. The reference type (Animal) has a reference to an object.
2. When sound() is called, the JVM looks at the actual object type.
3. The JVM calls the implementation of sound() in that actual class.

This process is called **Dynamic Method Dispatch** (Runtime Polymorphism)



Key Takeaway

Which method gets executed is decided at runtime, based on the actual object referred to by the reference variable.

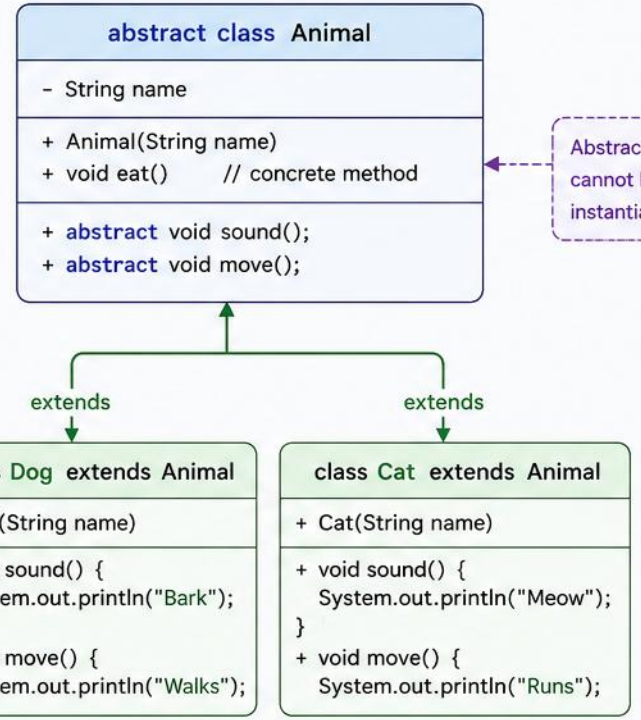
Dynamic Method Dispatch enables flexibility and extensibility in Java by allowing subclass methods to override superclass methods and be invoked at runtime based on the actual object.

ABSTRACT CLASSES IN JAVA

An abstract class in Java is a class that cannot be instantiated and is meant to be subclassed. It can have abstract (unimplemented) methods as well as concrete (implemented) methods.

KEY POINTS

- Declared using the **abstract** keyword.
- Cannot create objects of an abstract class.
- Can have **abstract methods** (without body) and **concrete methods** (with body).
- A subclass must **extend** the abstract class and **implement** all abstract methods.
- Supports abstraction and provides a common blueprint for subclasses.



EXAMPLE

```
abstract class Animal {
   String name;

   Animal(String name) {
     this.name = name;
   }

   void eat() {
     System.out.println(name + " eats");
   }

   abstract void sound();
   abstract void move();
}

class Dog extends Animal {
   Dog(String name) { super(name); }

   void sound() { System.out.println("Bark"); }
   void move()  { System.out.println("Walks"); }
}

class Cat extends Animal {
   Cat(String name) { super(name); }

   void sound() { System.out.println("Meow"); }
   void move()  { System.out.println("Wans"); }
}

// Usage
Animal a;          // reference
// Animal obj = new Animal("Tom"); // Error

Animal a1 = new Dog("Buddy");
a1.eat();   // Buddy eats
a1.sound(); // Bark
a1.move();  // Walks
```

HOW IT WORKS

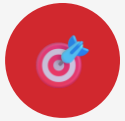
1. Abstract class defines a blueprint with some common behavior and abstract methods.
2. Subclasses extend the abstract class.
3. Subclasses provide implementation for all abstract methods.
4. Objects are created from subclasses and used via superclass reference.

BENEFITS

- Provides a common structure for related classes.
- Promotes code reusability and consistency.
- Achieves abstraction (hide implementation details).
- Supports the principle of “program to an interface”.

TRY IT YOURSELF — EXERCISE 4: ABSTRACT CLASSES

Reinforces: abstract classes and compiler-enforced contracts, as just covered.



1

Goal

Turn Account into an abstract class with one abstract method

2

File

AbstractAccount.java, AbstractSavings.java, AbstractDemo.java

3

Prove it

new AbstractAccount(...) fails to compile — abstract classes can't be instantiated

4

Compile & Run

javac *.java && java AbstractDemo

5

Reference

exercises/exercise-04-abstract-classes.md

TRY IT NOW Complete Exercise 4 before continuing — see exercises/exercise-04-abstract-classes.md.

INTERFACES IN JAVA

An interface is a blueprint of a class. It contains abstract methods (by default **public and abstract**) and **constants (public, static and final)**. A class implements an interface and provides the implementation of its methods.

Java

INTERFACES

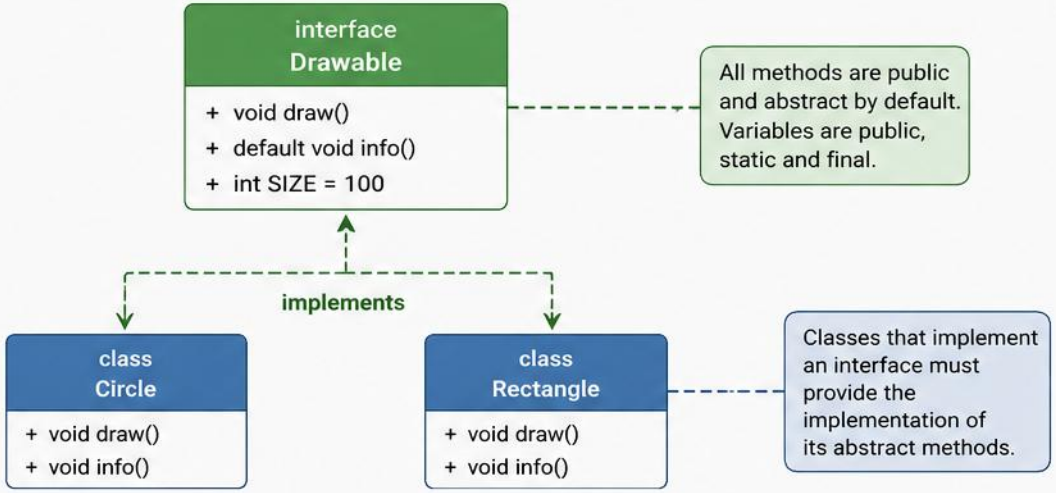
KEY POINTS

- Declared with the keyword **interface**.
- Cannot be instantiated.
- Methods are public and abstract by default.
- Variables are public, static and final by default.
- A class implements an interface using the keyword **implements**.

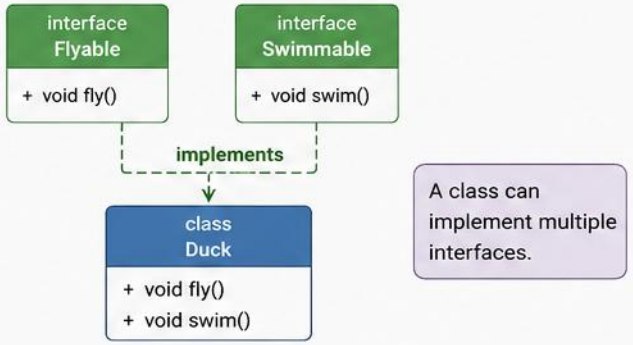
EXAMPLE

```
interface Drawable {
     void draw();                // abstract method
     default void info() {       // default method
          System.out.println("Drawable");
     }
     int SIZE = 100;             // public static final
}

class Circle implements Drawable {
    public void draw() {
         System.out.println("Drawing Circle");
    }
}
class Rectangle implements Drawable {
    public void draw() {
         System.out.println("Drawing Rectangle");
    }
}

// Usage
Drawable d1 = new Circle();
d1.draw();  // Drawing Circle
d1.info();  // Drawable
```

HOW INTERFACES WORK



MULTIPLE INHERITANCE WITH INTERFACES



BENEFITS

- Achieves 100% abstraction.
- Supports multiple inheritance.
- Promotes loose coupling.
- Enhances flexibility and scalability.
- Used to define common behavior across unrelated classes.

INTERFACES IN JAVA

Implementing an interface, and combining multiple interfaces.

```
public class Circle implements Drawable {
  private double radius;
  public Circle(double radius) {
    this.radius = radius;
  }
  @Override
  public void draw() {
    System.out.println("Drawing radius " + radius);
  }
}

Drawable d = new Circle(5.0);
d.draw();       // calls Circle's draw()
d.info();       // default method
Drawable.help(); // static method
```

```
interface A { void first(); }
interface B { void second(); }

class MyClass implements A, B {
  public void first() { ...; }
  public void second() { ...; }
}
```

✅ IMPORTANT POINTS

- All interface methods are public + abstract by default
- All interface variables are public static final (constants)
- A class can implement multiple interfaces — multiple inheritance of type

KEY TAKEAWAY A class must provide implementations for all abstract methods of the interfaces it implements.

INTERFACE VS ABSTRACT CLASS IN JAVA

Both achieve abstraction; the difference is purpose, capability, and usage.

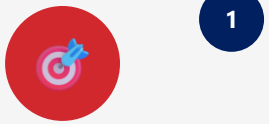
ASPECT	INTERFACE	ABSTRACT CLASS
Methods	Abstract by default; default/static (Java 8+)	Abstract + concrete; can have final methods
Variables	public static final only (constants)	Instance, static, or final variables
Constructors	Cannot have constructors	Can have constructors
Access modifiers	Methods public by default	Any modifier (public/protected/private)
Inheritance	Implement multiple (implements)	Extend only one (extends)
Purpose	Define a contract — what a class can do	Share code & state — what a class is

In short: Interface is for "what a class can do." Abstract class is for "what a class is."

KEY TAKEAWAY Use an interface for a capability/contract; use an abstract class to share code and state.

TRY IT YOURSELF — EXERCISE 5: INTERFACE PRACTICE

Reinforces: interfaces and implements, as just covered.



1

Goal

Create interface Printable { void printDetails(); } and implement it

2

File

Printable.java, Customer.java, InterfaceDemo.java in examples/module-03-exercises/

3

Key concept

implements Printable, then call printDetails() through a Printable reference

4

Compile & Run

javac *.java && java InterfaceDemo

5

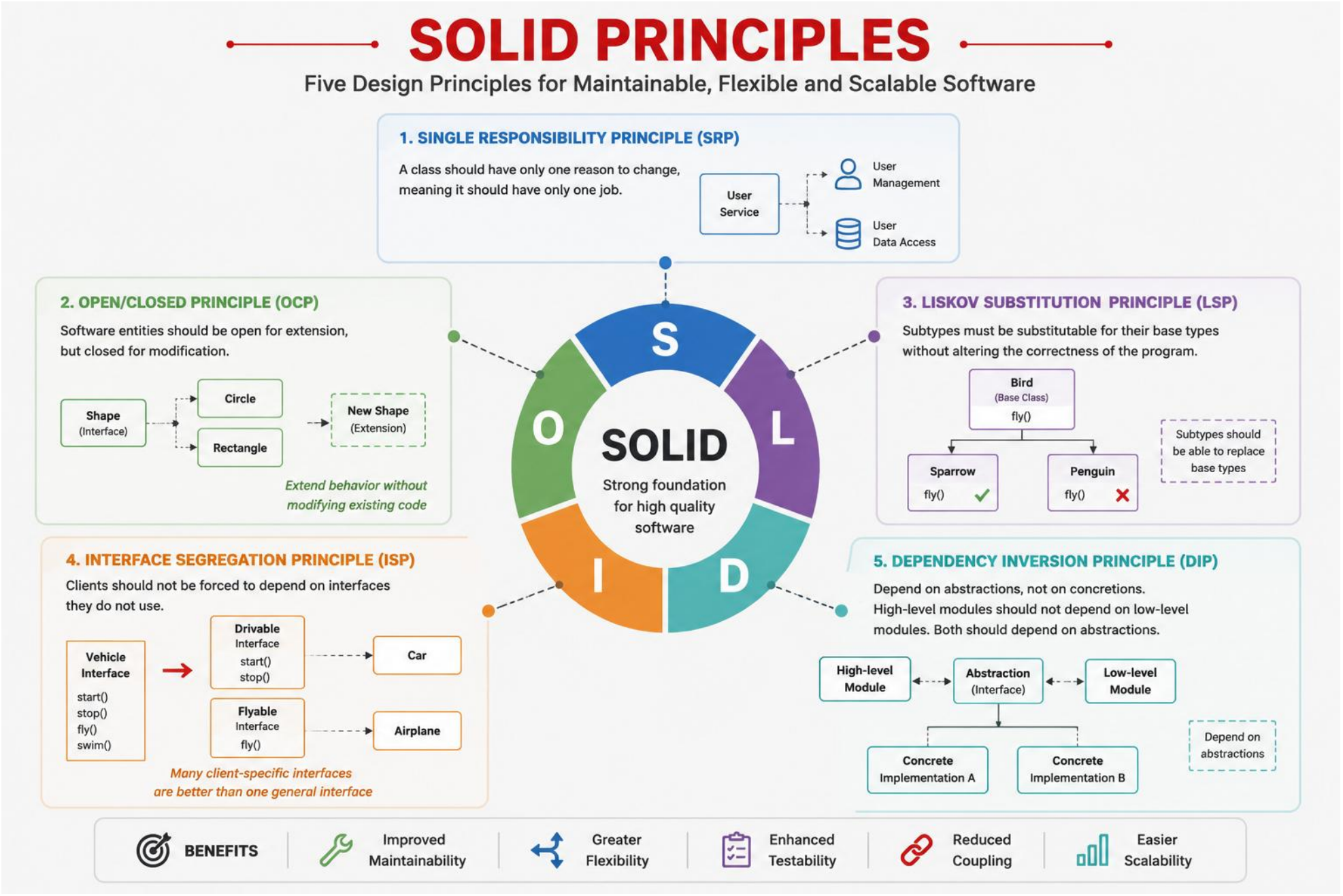
Reference

exercises/exercise-05-interface.md

TRY IT NOW Complete Exercise 5 before continuing — see exercises/exercise-05-interface.md.

SOLID PRINCIPLES

Five Design Principles for Maintainable, Flexible and Scalable Software



SINGLE RESPONSIBILITY PRINCIPLE (SRP)

A class should have only one reason to change — only one job or responsibility.

```
// ❌ Bad: one class, two responsibilities
class Employee {
  void setEmployee(int id, String name, double salary) {...}

  // Responsibility 2: generate report
  String generateReport() {
    return "ID:"+id+" Name:"+name;
  }
}
```

```
// ✅ Good: separate responsibilities
class Employee { ...getters... }

class EmployeeReportGenerator {
  String generateReport(Employee e) {
    return "ID:"+e.getId();
  }
}
```

❓ WHY IS SRP IMPORTANT?

- Easier maintenance — changes are localized
- Fewer bugs — smaller, focused classes are less error-prone
- Better testability — each class can be tested in isolation

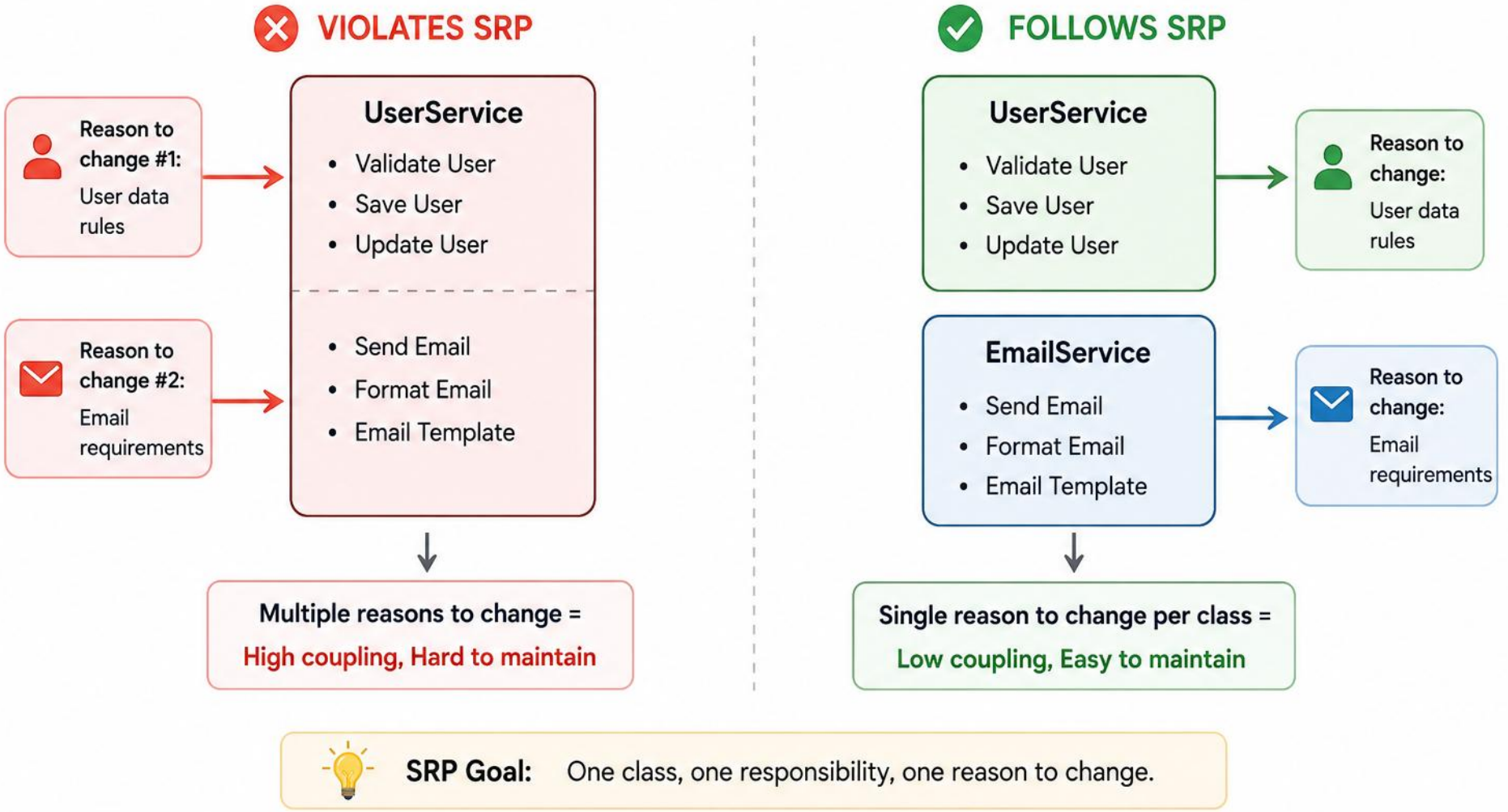
👩‍🍳 REAL-WORLD ANALOGY

A restaurant Chef cooks food; a Waiter takes orders and serves. Each has one job, one reason to change — just like well-designed classes.

KEY TAKEAWAY High cohesion in a class, low coupling between classes — the key to good design.

SINGLE RESPONSIBILITY PRINCIPLE

A class should have only one reason to change,
meaning it should have only one job or responsibility.



SRP Goal: One class, one responsibility, one reason to change.

TRY IT YOURSELF — EXERCISE 6: SOLID SPOT-CHECK

Reinforces: the Single Responsibility Principle you just saw.

1

Goal

Split one method that mixes business logic and System.out printing

2

Key concept

Separate calculate (logic) from display (printing) — one reason to change each

3

File

SolidDemo.java in examples/module-03-exercises/

4

Deliverable

A short note on what SRP improved about the method boundaries

5

Reference

exercises/exercise-06-solid-check.md

TRY IT NOW Complete Exercise 6 before continuing — see exercises/exercise-06-solid-check.md.

OPEN/CLOSED PRINCIPLE (OCP)

Software entities should be OPEN for extension, but CLOSED for modification.

```
// ❌ Violates OCP — must edit for every shape
class ShapeAreaCalculator {
  double area(String type, double r,
      double l, double w) {
    if (type.equals("CIRCLE"))
      return 3.14 * r * r;
    // add more if/else for each new shape...
  }
}
```

```
// ✅ Follows OCP — extend, don't modify
interface Shape { double area(); }
class Circle implements Shape {
  double r;
  public double area(){ return 3.14*r*r; }
}
// new shape = new class, zero edits above
```

🔑 KEY POINTS

- Do not modify existing, tested code
- Extend behavior using new classes
- Use abstraction (interfaces / abstract classes)
- Leads to loosely coupled, scalable code

✅ WHEN TO USE OCP?

When requirements are likely to change, features are added frequently, or the system needs long-term maintenance — e.g., in frameworks and libraries.

KEY TAKEAWAY Build systems that can grow without breaking what already works.

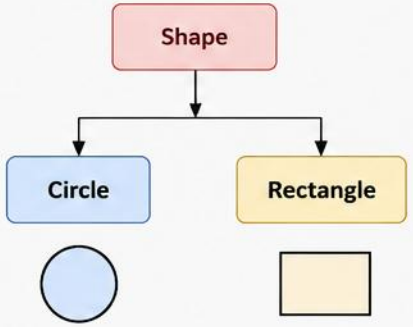
OPEN/CLOSED PRINCIPLE (OCP)

Software entities (classes, modules, functions) should be
open for extension, but **closed for modification**.

BEFORE: Not Closed for Modification

To add a new shape, we must modify existing code (Shape class).

```
class Shape {
  double area(Shape s) {
    if (s instanceof Circle)
       return 3.14 * s.r * s.r;
    else if (s instanceof Rectangle)
       return s.l * s.w;
       // To add new shape,
       // we must change this code!
  }
}
```

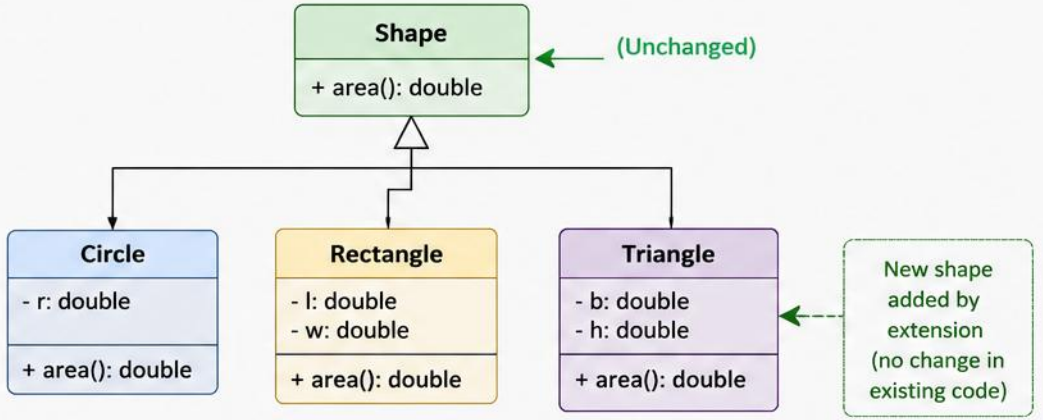


Problem

- ✗ Existing code changes when new shape is added.
- ✗ Higher risk of bugs.
- ✗ Violates OCP.

AFTER: Closed for Modification (Open for Extension)

We extend the behavior by adding new classes, without modifying existing code.



Client Code (Unchanged)

```
Shape s1 = new Circle(5);
Shape s2 = new Rectangle(4, 6);
Shape s3 = new Triangle(3, 4);  // New – no change in existing code
System.out.println(s1.area());
System.out.println(s2.area());
System.out.println(s3.area());
```

Benefits

- ✓ Closed for modification
- ✓ Open for extension
- ✓ Easier to maintain
- ✓ Less risk of bugs
- ✓ Follows OCP

Key Idea: Design your modules so that **new functionality can be added by extension** (new classes) **instead of modifying existing, tested code.**

LISKOV SUBSTITUTION PRINCIPLE (LSP)

Objects of a superclass should be replaceable with objects of its subclasses without breaking correctness.

```
// ❌ Violates LSP
class Rectangle {
  void setWidth(int w) { width = w; }
  void setHeight(int h) { height = h; }
}
class Square extends Rectangle {
  void setWidth(int w) { width=height=w; }
}  // breaks Rectangle's contract!
```

```
// ✅ Follows LSP — use abstraction
interface Shape { double area(); }
class Rectangle implements Shape {...}
class Square implements Shape {...}
// Both fully substitutable wherever Shape is expected
```

🔑 KEY POINTS

- Subclasses must be substitutable for their base class
- Subclass must honor the contract (behavior) of the base class
- Preserves correctness; prevents unexpected behavior
- A subclass should not break the rules defined by its superclass

🦜 REAL-WORLD ANALOGY

A Bird superclass with fly(): if Penguin extends Bird and can't fly, substitution breaks. Use an abstraction that only defines what all birds truly share.

KEY TAKEAWAY Design subtype hierarchies carefully — when in doubt, prefer composition over inheritance.

LISKOV SUBSTITUTION PRINCIPLE (LSP)

Objects of a subclass should be replaceable with objects of the superclass without affecting the correctness of the program.

SUPERCLASS

Rectangle

+ area(): double
+ setWidth(w: double): void
+ setHeight(h: double): void

SUBCLASS (Good)

Square

+ area(): double
+ setWidth(w: double): void
+ setHeight(h: double): void

Respects LSP

SUBCLASS (Bad)

RectangleWithFixedWidth

+ area(): double
+ setWidth(w: double): void
// Not allowed to change width
+ setHeight(h: double): void

Violates LSP

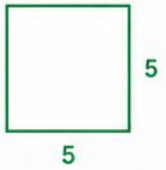
USING THE SUPERCLASS

```
void printArea(Rectangle r) {
   r.setWidth(5);
   r.setHeight(4);
   System.out.println(r.area());
}
```

Works Correctly

```
Rectangle r = new Square();
printArea(r);
```

Output: 20.0



Fails (Unexpected)

```
Rectangle r = new RectangleWithFixedWidth();
printArea(r);
```

Output: Incorrect!

Width cannot be changed, behavior breaks.

LSP in Action: A subclass must fully honor the contract of the superclass so that it can be used interchangeably without altering the desired behavior of the program.

INTERFACE SEGREGATION PRINCIPLE (ISP)

Clients should not be forced to depend on interfaces they do not use.

```
// ❌ Fat interface forces unused methods
interface Worker {
  void work(); void eat(); void drive();
}
class HumanWorker implements Worker {
  public void drive() {
    throw new UnsupportedOperationException();
  }  // Not all workers can drive!
}
```

```
// ✅ Small, focused interfaces
interface Workable { void work(); }
interface Eatable  { void eat(); }
interface Drivable { void drive(); }
class HumanWorker implements
    Workable, Eatable { ...; }
```

🔑 KEY POINTS

- Create small, focused interfaces for specific client needs
- No client should be forced to implement methods it doesn't use
- Reduces coupling between unrelated interfaces and clients
- Improves maintainability and readability

🍽️ REAL-WORLD ANALOGY

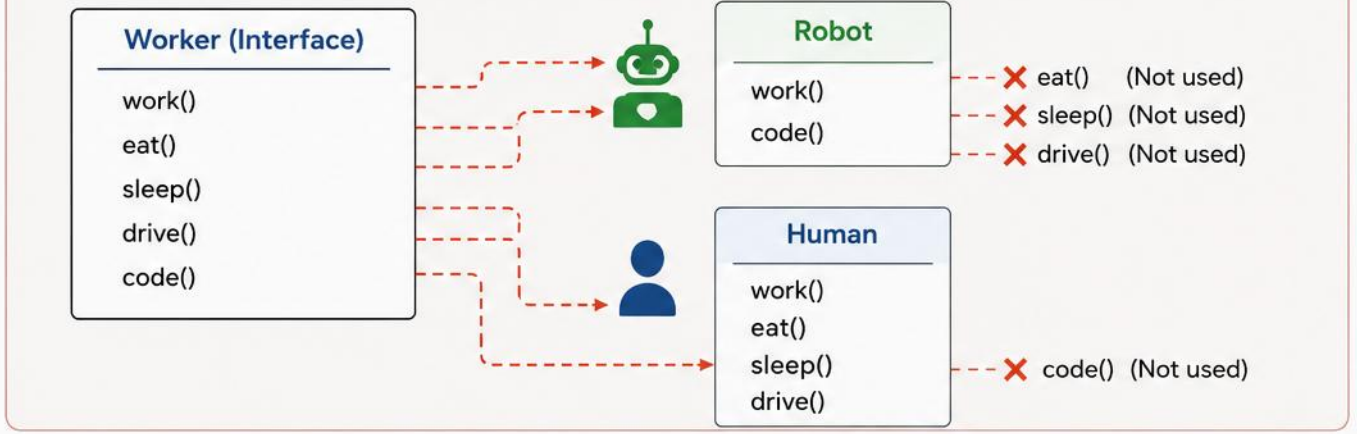
A waiter doesn't need to cook. Give chefs a Cooking interface and waiters a Serving interface — not one giant "restaurant worker" interface.

KEY TAKEAWAY Many small, client-specific interfaces beat one large, general-purpose interface.

INTERFACE SEGREGATION PRINCIPLE (ISP)

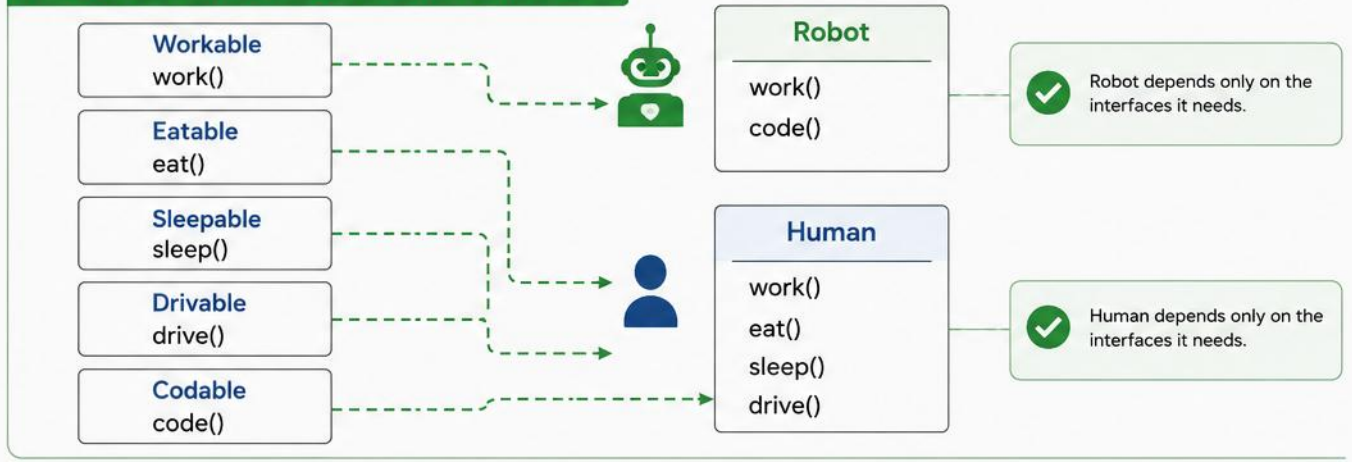
Clients should not be forced to depend on interfaces they do not use.

PROBLEM: FAT INTERFACE



- The Worker interface forces Robot to depend on methods it does not use (eat, sleep, drive).
- Any change in the unused methods can impact the clients unnecessarily.
- Leads to bloated interfaces and tighter coupling.

SOLUTION: SEGREGATED INTERFACES



- Clients are not forced to depend on methods they do not use.
- Interfaces are focused and cohesive.
- Easier to maintain, extend and test.
- Reduces impact of changes and improves flexibility.

Depends on (unused methods included)

Depends on (only required methods)

DEPENDENCY INVERSION PRINCIPLE (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.

```
// ❌ Tight coupling to a concrete class
class MySQLDatabase { void save(){} }
class UserService {
  private MySQLDatabase db =
      new MySQLDatabase();
  // swapping DB means editing this class
}
```

```
// ✅ Depend on an abstraction
interface Database { void save(String d); }
class MySQLDatabase implements Database {...}
class UserService {
  private Database db;
  UserService(Database db){ this.db=db; }
}
```

🔑 KEY POINTS

- Depend on abstractions, not concrete classes
- High-level and low-level modules both depend on abstractions
- Reduces coupling between components
- Use Dependency Injection to achieve DIP

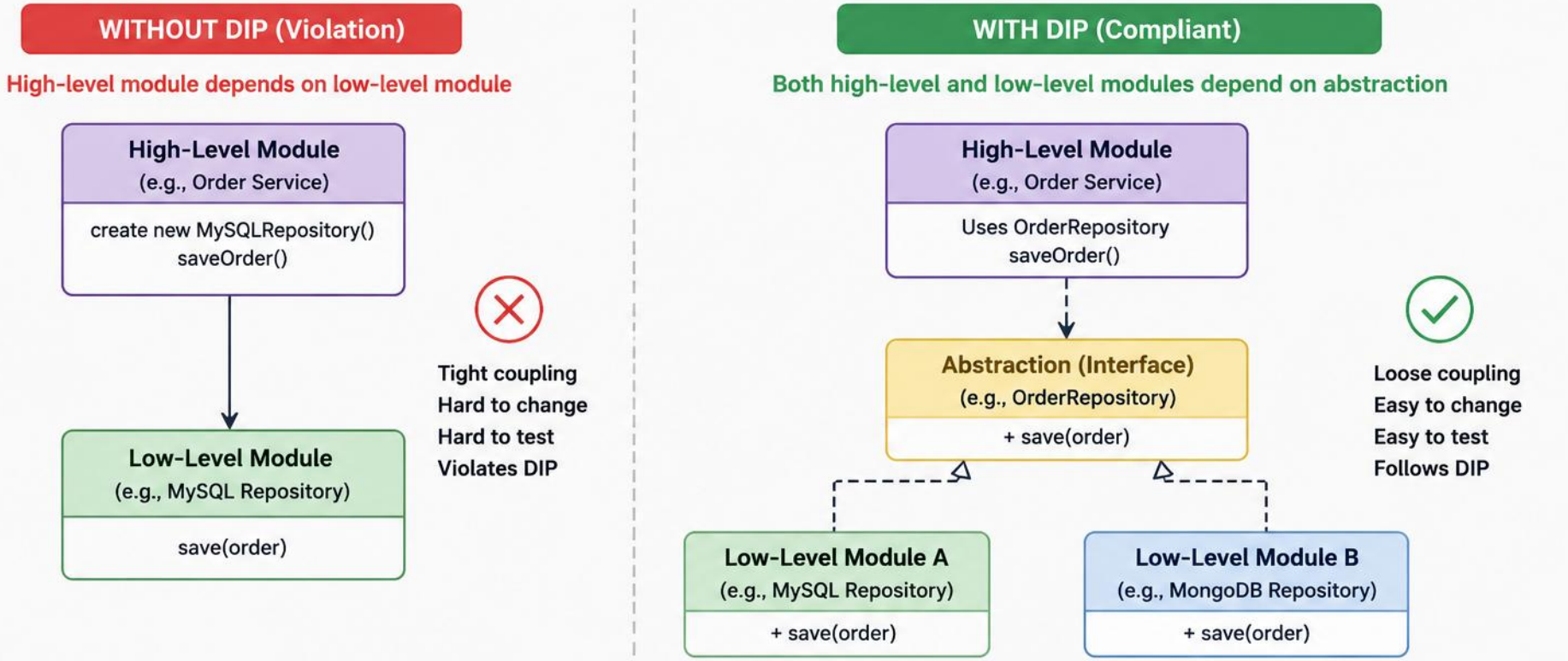
💡 REAL-WORLD ANALOGY

A light switch doesn't depend on one specific bulb — it depends on an electrical interface. Any bulb (LED, CFL) can be plugged in.

KEY TAKEAWAY Program to an interface, not to an implementation. Abstractions are stable; details can change.

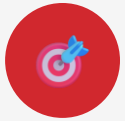
DEPENDENCY INVERSION PRINCIPLE (DIP)

High-level modules should not depend on low-level modules. Both should depend on abstractions.
Abstractions should not depend on details. Details should depend on abstractions.



TRY IT YOURSELF — EXERCISE 7: SOLID BEYOND SRP

Reinforces: OCP, LSP, ISP, and DIP, as just covered.



1

Goal

Spot-check the other four SOLID principles using the Account hierarchy

2

New file

FrozenAccount.java — an LSP-safe subclass with no special-casing

3

Write

One sentence each for OCP, LSP, ISP, and DIP

4

Key concept

A subclass must honor its parent's contract, not just compile

5

Reference

exercises/exercise-07-solid-beyond-srp.md

TRY IT NOW Complete Exercise 7 before continuing — see exercises/exercise-07-solid-beyond-srp.md.

OOP DESIGN PATTERNS

Reusable solutions to common object-oriented design problems.

DESIGN PATTERNS

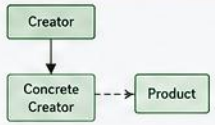
Proven, reusable approaches to solve recurring problems in software design. Improve code reusability, flexibility, and maintainability.

CREATIONAL PATTERNS

Deal with object creation mechanisms.

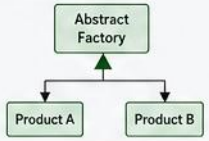
Singleton

Ensure a class has only one instance and provide a global point of access to it.



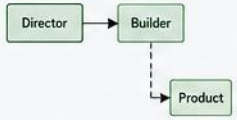
Factory Method

Define an interface for creating an object, but let subclasses decide which class to instantiate.



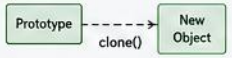
Abstract Factory

Provide an interface for creating families of related or dependent objects without specifying their concrete classes.



Builder

Separate the construction of a complex object from its representation so that the same construction process can create different representations.



Prototype

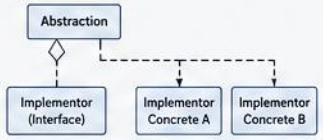
Create new objects by cloning an existing object, known as the prototype.

STRUCTURAL PATTERNS

Deal with object composition and structure.

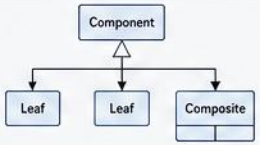
Adapter

Convert the interface of a class into another interface clients expect.



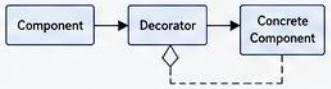
Bridge

Decouple an abstraction from its implementation so that the two can vary independently.



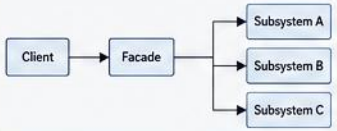
Composite

Compose objects into tree structures to represent part–whole hierarchies.



Decorator

Attach additional responsibilities to an object dynamically.

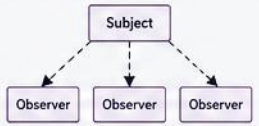


Facade

Provide a unified interface to a set of interfaces in a subsystem. Facilitate communication and ease of use.

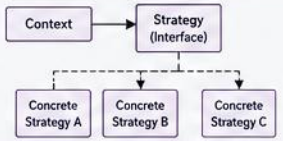
BEHAVIORAL PATTERNS

Deal with communication between objects.



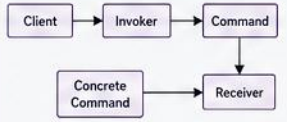
Observer

Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified.



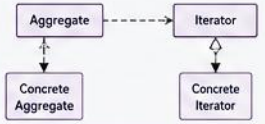
Strategy

Define a family of algorithms, encapsulate each one, and make them interchangeable.



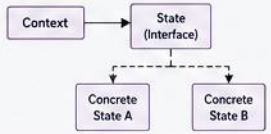
Command

Encapsulate a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undoable operations.



Iterator

Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.



State

Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

BENEFITS • Reusability • Maintainability • Flexibility • Scalability • Best Practices

PRINCIPLE Program to an interface, not an implementation.

OOP DESIGN PATTERNS IN ENTERPRISE APPLICATIONS

Three categories of reusable design solutions — and what each is for. (1 of 2)

Creational — Object creation

Structural — Object composition

Behavioral — Communication & responsibility

Creational

Object creation mechanisms — flexible, decoupled ways to build objects.

INCLUDES
Singleton, Factory Method, Abstract Factory, Builder, Prototype

Structural

Object composition — compose classes/objects into larger, flexible structures.

INCLUDES
Adapter, Facade, Decorator, Composite, Flyweight, Proxy

Behavioral

Object communication — govern how objects interact and share responsibility.

INCLUDES
Strategy, Observer, Command, Template Method, State, Chain of Responsibility, Mediator

KEY TAKEAWAY Three categories organize every pattern: Creational (object creation), Structural (object composition), and Behavioral (object communication).

OOP DESIGN PATTERNS IN ENTERPRISE APPLICATIONS

The six patterns most worth knowing by name in everyday Java work. (2 of 2)

Adapter

Lets incompatible interfaces work together.

Use: 3rd-party libraries, Legacy systems

Facade

Simplified interface to a complex subsystem.

Use: Complex modules, Layered architectures

Factory Method

Lets subclasses decide which class to instantiate.

Use: Plugins, Notification systems

Builder

Separates complex object construction from representation.

Use: Reports, Documents, Queries

Strategy

Encapsulates interchangeable algorithms.

Use: Payment methods, Pricing, Sorting

Observer

One-to-many notification when state changes.

Use: Event handling, Notifications

KEY TAKEAWAY Factory, Builder, Adapter, Facade, Strategy, and Observer — six patterns that solve recurring design problems and show up constantly in enterprise Java code.

MODELING REAL-WORLD BUSINESS ENTITIES

Accurate models of business entities are the foundation of maintainable enterprise apps. (1 of 2)

❓ WHAT IS AN ENTITY?

A distinct, identifiable object or concept in the business domain that has a lifecycle and holds important data.

- Has a unique identity
- Has attributes (data)
- Has behavior (operations)
- Has relationships with other entities

🛒 EXAMPLE: E-COMMERCE ORDER

Customer

Places orders

Order

Contains order items

Payment

Payment for an order

Shipment

Delivery details

📐 KEY MODELING GUIDELINES

- Identify entities — use nouns from business language
- Define clear responsibilities for each entity
- Use value objects for descriptive, immutable data
- Model relationships & multiplicities explicitly
- Enforce business rules close to the data

KEY TAKEAWAY Good domain models capture the heart of your business — they make code easier to understand and extend.

MODELING REAL-WORLD BUSINESS ENTITIES IN JAVA

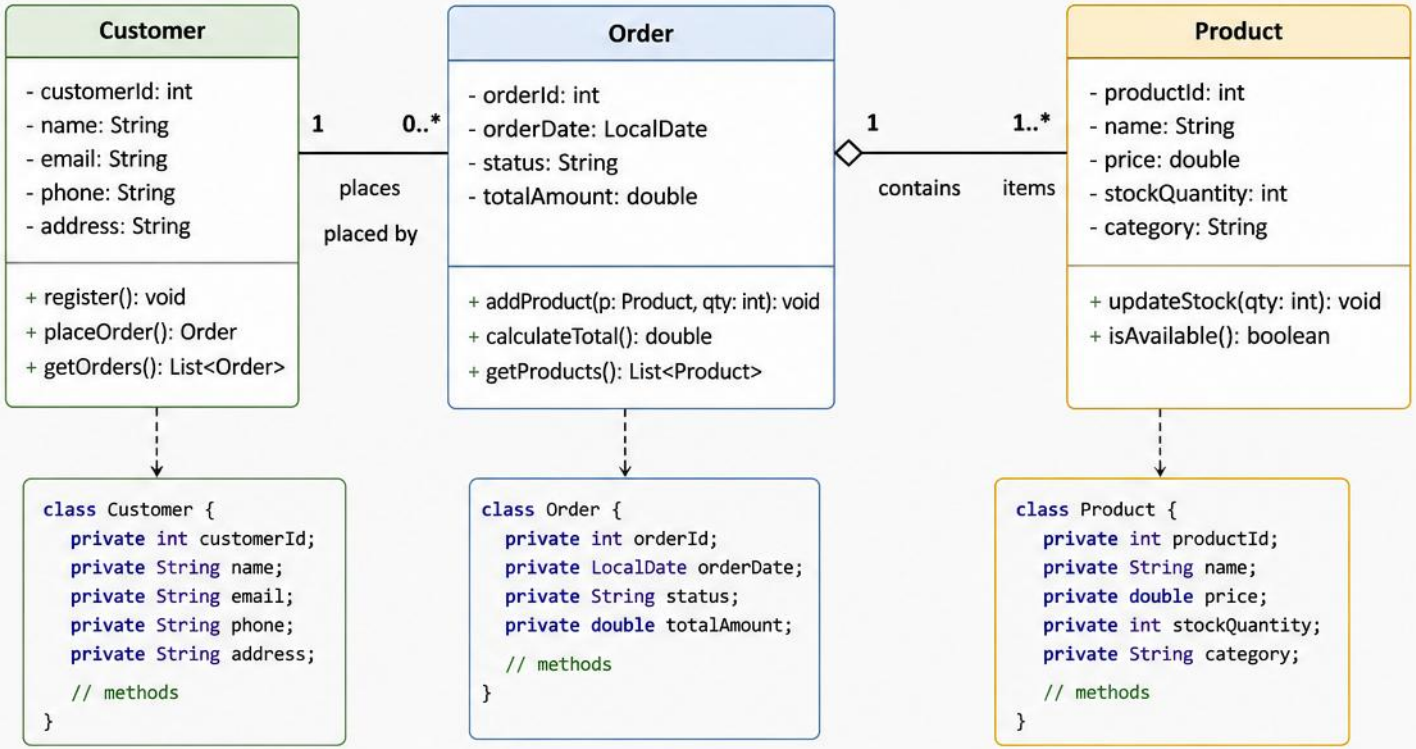
Represent business concepts as classes and define their attributes, behaviors, and relationships.

REAL-WORLD SCENARIO

An online store manages **Customers** who place **Orders**. Each Order contains one or more **Products**.

HOW WE MODEL IT

1. Identify Entities → Customer, Order, Product
2. Define Attributes → Properties of each entity
3. Define Behaviors → Actions each entity can perform
4. Define Relationships → How entities are associated



UML NOTATION GUIDE

Class
Represents an entity
- attribute: Type
Private attribute
+ method(): Type
Public method

Association
Relationship between entities

1 One

0..* Zero or many

Aggregation
Whole–part relationship
(Order contains Products)

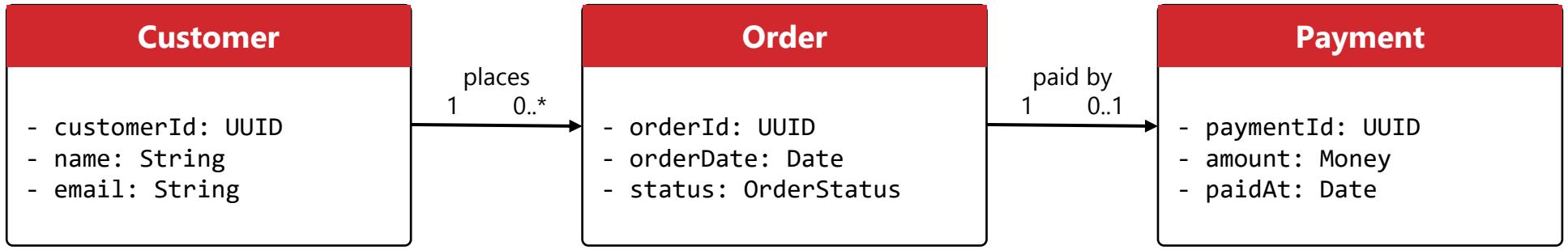
BENEFITS

- Maps real-world concepts to code
- Creates a clear structure
- Improves maintainability & scalability
- Facilitates communication between business & technical teams

Good modeling is the foundation of good software design.

DOMAIN MODEL: UML CLASS DIAGRAM

Customer, Order, and Payment — modeled with associations and multiplicities. (2 of 2)



📐 DESIGN NOTES

- A Customer can place many Orders (1-to-many)
- An Order is paid by zero or one Payment (0..1, since payment is optional until an order is confirmed)
- Payment stores the amount and paidAt timestamp for that order

✅ BENEFITS IN ENTERPRISE APPS

- Shared understanding — ubiquitous language across teams
- Maintainability — clear models reduce complexity
- Data integrity — business rules protected in the model

KEY TAKEAWAY Model relationships and multiplicities explicitly; keep the model simple and evolve it iteratively.

TRY IT YOURSELF — EXERCISE 8: MINI UML

Reinforces: the class diagram notation you just saw.

1

Goal

Draw Account, SavingsAccount, and Customer with their relationships

2

Deliverable

Paper or Mermaid diagram — no code required

3

Show

Inheritance (SavingsAccount → Account) and association (Customer → Account)

4

Key concept

Note multiplicities — one Customer can have many Accounts (1 : many)

5

Reference

exercises/exercise-08-uml-mini.md

TRY IT NOW Complete Exercise 8 before continuing — see exercises/exercise-08-uml-mini.md.

PRE-LAB EXERCISES OVERVIEW

Eight short warm-ups to practice before the full Lab 3 build.

🎯 PURPOSE

Eight short, focused exercises that let you practice one OOP concept at a time — entities, encapsulation, inheritance, abstract classes, interfaces, UML, and SOLID — using a simple banking domain, before you start the full Lab 3 build.

🔍 HOW TO USE THEM

- Work through them in order, after the module slides
- Keep practice code separate from the graded Lab 3 submission
- Each one takes minutes, not hours

📝 THE EIGHT EXERCISES

Located in labs/module-03/exercises/ — complete all eight before opening the full Lab 3 guide.

- **Exercise 1 —** List entities and one responsibility each
- **Exercise 2 —** Private balance with deposit/withdraw
- **Exercise 3 —** Override withdrawal across two subclasses
- **Exercise 4 —** Make Account abstract with one method
- **Exercise 5 —** Define and implement a simple interface
- **Exercise 6 —** Split calculation from display logic
- **Exercise 7 —** Spot-check the other four principles
- **Exercise 8 —** Sketch a small class diagram

✅ READY FOR LAB 3?

- Each exercise compiles and runs (or notes are complete for analysis-only tasks)
- You can explain each result in one sentence
- Practice files are kept separate from the Lab 3 submission
- You completed all eight exercises in order
- You are comfortable with entities, encapsulation, and inheritance
- You are ready to open the full Lab 3 guide

KEY TAKEAWAY Eight quick exercises build the muscle memory you will need for the full Lab 3 Banking Management System build.

LAB OVERVIEW: BANKING MANAGEMENT SYSTEM

Build a menu-driven Banking Management System that applies core OOP pillars.

🎯 LAB OBJECTIVE

Build a menu-driven Banking Management System (package com.academy.bank) that applies encapsulation, inheritance, abstraction, interfaces, and polymorphism using plain JDK — no frameworks, no database.

🔍 YOU WILL PRACTICE

- Abstract class hierarchy: Account → Savings/Current
- Interface contract: Printable
- Polymorphic display via an Account[] array

🏦 BUSINESS DOMAIN

Bank staff need a console tool to create customers, open Savings or Current accounts, deposit and withdraw funds, and display balances — no database, just plain JDK.

- **Customer —** Profile; implements Printable
- **Account —** Balance, deposit/withdraw
- **Savings —** Interest + display
- **Current —** Fee + display
- **Transaction —** Deposit/withdraw log
- **Service —** Arrays + operations

📋 KEY REQUIREMENTS

- Create customers and open Savings or Current accounts
- Deposit and withdraw with balance validation
- Display accounts and customers polymorphically
- Keep Main thin — operations live in BankService
- Compile with javac -d out and run with java -cp out
- Sketch a UML class diagram of the design

KEY TAKEAWAY Build a menu-driven Banking Management System — encapsulation, inheritance, interfaces, and polymorphism working together.

LAB TASKS AND DELIVERABLES

Build the Account hierarchy, service layer, and menu for the Banking Management System.

📝 LAB TASKS (IN SEQUENCE)

1. Create Project & Package — set up the com.academy.bank folder structure
2. Define Printable & Customer — interface contract and customer profile
3. Build Abstract Account — shared fields, deposit()/withdraw(), abstract displayAccount()
4. Create SavingsAccount & CurrentAccount — interest vs. transaction fee, overridden display
5. Add Transaction — record deposit/withdraw activity for each account
6. Build BankService — arrays plus create/deposit/withdraw/display operations
7. Create Menu-Driven Main — thin entry point that delegates to BankService
8. Compile, Run & Document — verify polymorphic display, sketch UML, apply SOLID checklist

📦 EXPECTED DELIVERABLES

- **Source Code**
  Full com.academy.bank sources: Customer, Account, Savings, Current, Printable, Transaction, BankService, Main
- **Screenshots**
  Create customer/account, deposit, withdraw, polymorphic display, exit
- **UML Class Diagram**
  Inheritance, interface, and "uses" relationships
- **Design Note**
  SOLID principles and inheritance/polymorphism explained in your own words
- **Compile/Run Evidence**
  javac -d out and java -cp out commands with output

KEY TAKEAWAY 100-mark rubric: class design 20, inheritance/interface 25, polymorphism/service 25, menu/compile 15, UML/quality/evidence 15.

MODULE 3 SUMMARY — WHAT WE ACHIEVED

We learned to design real-world software using OOP principles, SOLID, and design patterns.

Understand

Real-world business entities and their behavior.

Design

Robust and maintainable class models.

Implement

Clean, modular and extensible code using OOP.

Apply

Design patterns and SOLID principles for better solutions.

OOP DESIGN PROCESS

1. Analyze Requirements
2. Identify Entities
3. Model Relationships
4. Design Class Diagram
5. Apply SOLID & Patterns
6. Implement & Test
7. Review & Improve

KEY TAKEAWAYS

- Good design starts with understanding the business
- Clear models lead to better code and fewer defects
- SOLID principles guide flexible, maintainable software
- Design patterns provide tested solutions to common problems
- Continuous improvement and refactoring keep systems healthy
- Interfaces and abstract classes give you two complementary tools for abstraction
- Good design means the right code, not just more of it

KEY TAKEAWAY Strong design leads to systems that are easy to extend, test, understand — and aligned with business goals. Next up: Exception Handling and File I/O.

MODULE 3 KNOWLEDGE CHECK

Test your understanding of key concepts from Module 3. (1 of 2)

1

Which is the best definition of an entity?

✅ **A distinct object/concept with a lifecycle and data**

2

Which UML diagram shows classes, attributes, methods & relationships?

✅ **Class Diagram**

3

In UML, which symbol represents composition?

✅ **Filled diamond**

4

Which SOLID principle: a class should have one reason to change?

✅ **Single Responsibility Principle**

5

Which pattern provides a simplified interface to a complex subsystem?

✅ **Facade**

KEY TAKEAWAY Choose the best answer for each question — questions 6–10 continue on the next slide.

MODULE 3 KNOWLEDGE CHECK

Test your understanding of key concepts from Module 3. (2 of 2)

6

Which principle lets you add functionality without modifying existing code?

✅ **Open/Closed Principle**

7

Customer places many Orders — which relationship describes this?

✅ **Association (1 to Many)**

8

Which principle: clients shouldn't depend on interfaces they don't use?

✅ **Interface Segregation Principle**

9

Which pattern notifies dependents automatically when one object changes?

✅ **Observer**

10

Primary focus when designing class models for enterprise apps?

✅ **Reflect the real-world business domain accurately**

📊 **SCORING GUIDE**

9–10: Excellent • 7–8: Good • 5–6: Needs Improvement • Below 5: Review the module again

KEY TAKEAWAY Practice, apply, and refactor your designs. Great design leads to great software.

Nicely done

You can design, build, and reason about object-oriented Java programs.

SOLID principles, design patterns, and real domain modeling are now part of your toolkit.